Searching and retrieving along-track altimetry

a sub-granule index on a discrete global grid, and the engines that build and use it

the aicesat design, from the data up

Kevin W. Beam

2026-10-18

Abstract

We describe the characteristics of along-track, non-gridded altimetry data that make it difficult to search and retrieve: data from ICESat's GLAS, Operation IceBridge's ATM, ICESat-2's ATLAS, GEDI, and a ground traverse, as they are archived in the Earthdata cloud. Using a canonical set of searches and retrievals based on geospatial area, temporal specification, and filtering and aggregating constraints, we show where granule-level search fails on such data and derive, from first principles, what an efficient system must record. We build a system that transforms granules and their metadata into an *index into the raw data*. Taking into account the logical and physical organisation of the files, we show how a discrete global grid system turns that index into a table that answers spatial searches exactly and cheaply, and whose rows are request specifications: the byte ranges that hold the matching points, retrievable from the granule files directly. We then describe the request engine that executes those specifications and builds a local data lake from which analysis and visualisation proceed, and the framework that lets each collection be a value plugged into one shared pipeline. Every number is a measurement on the deployed instance; the margin records, chapter by chapter, which parts exist today.

How to read this document

The report proceeds in eight parts and is meant to be read in order. Part I characterises the data: what an along-track product is, how the archive packages it into granules, and how the bytes lie inside a granule. Part II states the problem: the queries the system is asked, where granule-level search fails on these data (measured), and the seven requirements that follow. Part III is about discrete global grids: what they are, four of them compared, and how the chosen one is used. Part IV derives the index as a table and fixes what a row means. Part V is a functional design for the builder that writes it, Part VI for the path from a query specification to a plan to its execution, and Part VII for the retrieval engine and the lake it fills. Part VIII is the framework: how six collections share one pipeline, how configuration and the grid become parameters, how the design is enforced in a language that enforces nothing. Each chapter from Part IV on opens by naming the requirement it discharges. Parts V–VII separate the *structural* view (which values and functions exist) from the *runtime*

view (which processes and threads run them) and the *computation* (what work is done per point, per chunk, per granule).

Figures use one grammar throughout (Figure 1). The margin carries two kinds of note: numbered asides, and unnumbered notes in italics saying what already exists in the code for that chapter. Names of modules and functions live in those notes and in the listings, so the body can stay with concepts. Citations are numbered; every number in the text is a measurement on the deployed instance unless a citation says otherwise.

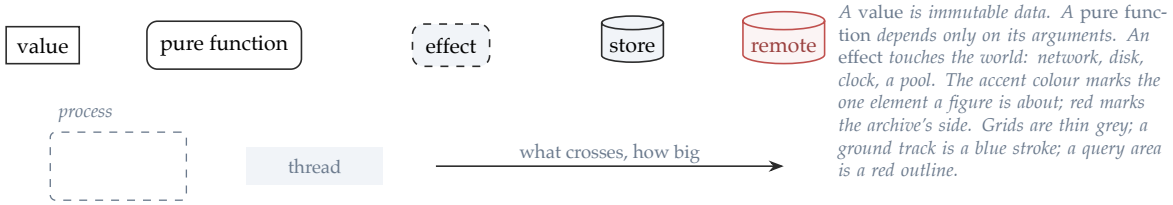


Figure 1: The figure grammar. Sharp corners are data; rounded corners are code; grey dashed code has side effects; cylinders are stores, red when they belong to the archive. Arrows say what crosses and, where measured, how much.

Contents

I The data 3

1 Along-track, non-gridded data 4

2 Logical organisation: the granule and the product 6

3 Physical organisation: how the bytes lie 7

II The problem 11

4 A canonical set of queries 11

5 Search and retrieval today, and how they fail 13

6 Requirements from first principles 14

III Discrete global grids 17

7 What a discrete global grid system is 17

8 Four systems 18

9 The grid in this system 20

IV The index 23

10 The index as a relation 23
10.1 The run 23
10.2 Photons: runs with edges 24

11 The index as files 25

V A functional design for the index builder 27

12 Build as a pipeline of values 27

13 Reading one granule 28

14 The build at runtime 29

VI From query specification to plan to execution 32

15 The query specification 32

16 The plan 32

17 Executing the plan: one chunk, step by step 33

VII The retrieval engine and the lake 36

18 The lake as a memo 36

19 What reads the lake 37

20 The query at runtime 38

VIII The framework 40

21 The collection descriptor 40

22 Configuration as a value 42

23 The grid as a parameter 43

24 Replay 44

25 Packages and the structural view 45

26 Purity in Python, and what not to do 46

Appendices 47

A Migration path 47

B Test strategy 47

C Measurements the design leans on 48

D Glossary 48

References 52

Part I

The data

1 Along-track, non-gridded data

A laser altimeter measures the time of flight of a pulse from the instrument to the ground and back. Combined with the instrument's position and pointing, each pulse gives one *footprint*: a position on the ellipsoid, a height, and a time. The instrument fires continuously while the platform moves, so the measurements form a time-ordered sequence along a *ground track*. Nothing about the sequence is a grid. Consecutive footprints are metres to hundreds of metres apart along the track; the next track is kilometres away or, for a repeat pass, offset by whatever the pointing achieved that day. The position of a measurement is *data stored with it*, not something implied by its place in an array.

That one property is the source of every difficulty this report is about. In a raster, the question "which elements lie inside this box" is arithmetic on indices. In an along-track product, a point's position is known only after the point has been read, so the question has no answer short of reading everything, unless someone has read everything once and written the answer down.

The instruments. Five sources of along-track data enter the system (Table 1, Figure 2).

- **ICESat / GLAS** (2003–2009) carried a single-beam 1064 nm altimeter firing at 40 Hz, giving footprints of roughly 70 m every ∼172 m along track [1]. Early laser failures forced operation in 33-day campaigns, three a year; the record is 19 campaigns, L1A through L2F, so coverage is intermittent in time by design. We use the GLAH06 land-elevation product, release 34 [2].
- **Operation IceBridge / ATM** (2009–2019) flew the Airborne Topographic Mapper, a conically scanning lidar, on survey aircraft to bridge the gap between the two satellites [3]. The Level-2 "icessn" product resamples the dense scan into platelets: a plane is fitted to blocks of points at regular intervals along and across the swath; each record is an 80 m across-track nadir platelet, three or five outer platelets, and about half a second of flight [4, 5]. The result is a short, dense, irregular set of flight lines rather than an orbit.
- **ICESat-2 / ATLAS** (2018–) splits a 532 nm beam into three pairs; each pair is a strong and a weak beam (energy ratio about 4:1) 90 m apart, and the pairs are 3.3 km apart across track [6]. The photon-counting detector records individual returned photons, about one every 0.7 m along track. ATL03 is the geolocated photon product [7, 8]; ATL06 fits 40 m segments to the photons, posted every 20 m, and reports one land-ice height per segment [9, 10]. The 1 387 reference ground tracks repeat every 91 days.
- **GEDI** (2019–) is a lidar on the International Space Station: three lasers, one split into two coverage beams, dithered into eight ground tracks about 600 m apart, each with 25 m footprints every

source	product	years	footprint	along-track	beams or tracks	latitude
ICESat GLAS	GLAH06 r34	2003–09	∼70 m	∼172 m	1	±86°
IceBridge ATM	ILATM2 v2	2009–19	80 m platelet	∼0.5 s of flight	nadir + 3–5 outer	60–90°N, 53–90°S
ICESat-2 ATLAS	ATL03 v7	2018–	∼17 m	∼0.7 m (photons)	3 pairs, strong/weak	±88°
ICESat-2 ATLAS	ATL06 v7	2018–	40 m segment	20 m posting	3 pairs, strong/weak	±88°
GEDI	GEDI02_A v3	2019–	25 m	∼60 m	8 tracks, full/coverage	±51.6°
Summit GPS	IS2TGPSSS v1	2006–	point	continuous	1 transect	Summit only

Table 1: The along-track sources in the system. The latitude column is the declared footprint the system uses to decide whether a collection can have data over an area at all; the years are the default search windows.

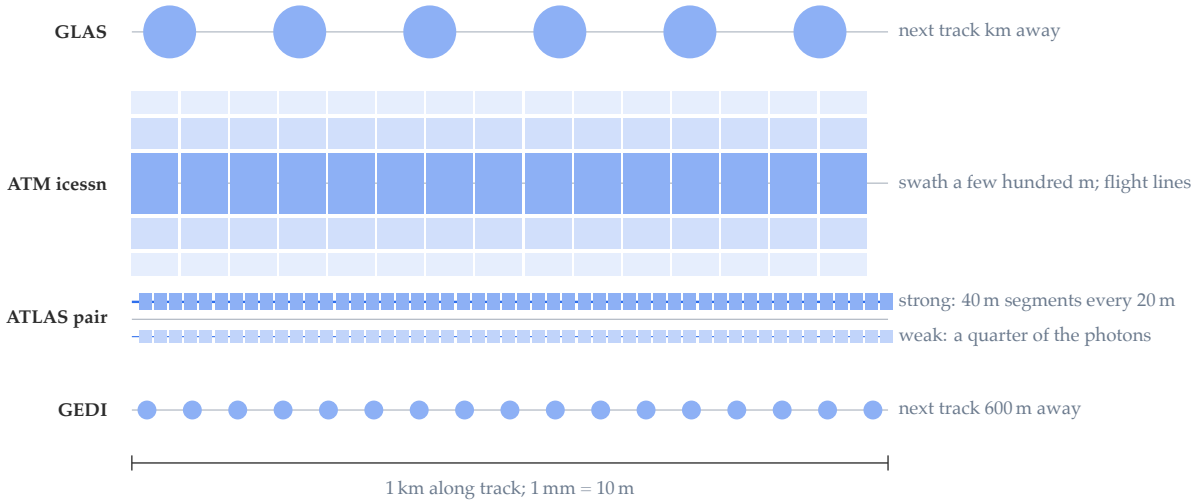


Figure 2: One kilometre of each instrument's sampling at one scale. The products differ by three orders of magnitude in along-track density and by as much across track, but all of them are sequences of positioned measurements along a line, and none of them is a grid.

∼60 m [11, 12]. The station's orbit confines it to ±51.6°, so it never sees an ice sheet; it enters the system for mountain glaciers and as a test that the design is not ice-specific. We use the L2A footprint product, version 3 [13].

- **The Summit traverse** is ground truth rather than altimetry: a kinematic GPS antenna on a sled towed monthly along a 15 km transect west of Summit Station, Greenland, laid along ICESat-2 reference tracks 749 and 879 [14]. It has the along-track shape of an altimetry product and none of its scale.

What they share. Every source delivers positions, heights and times as parallel arrays in time order, with quality indicators alongside; every source is sparse across track and continuous along it; and every source has holes. GLAS returns from cloud tops sit kilometres above the surface; its own cloud flag is set on 99.9 % of shots and so distinguishes nothing, while about 3 % of shots are gross cloud returns. ATLAS loses the surface under thick cloud and over wet firn the 532 nm signal penetrates the snow. GEDI's accuracy degrades with slope. Fill values mark where an algorithm declined to report: GLAS uses 1.798×10^{308} where a reader expects NaN, ATL06 uses a large positive float. Any system that indexes these data must therefore record where points *are* rather than where the track *was*, and must be prepared for a granule that crosses an area to contribute nothing to it.

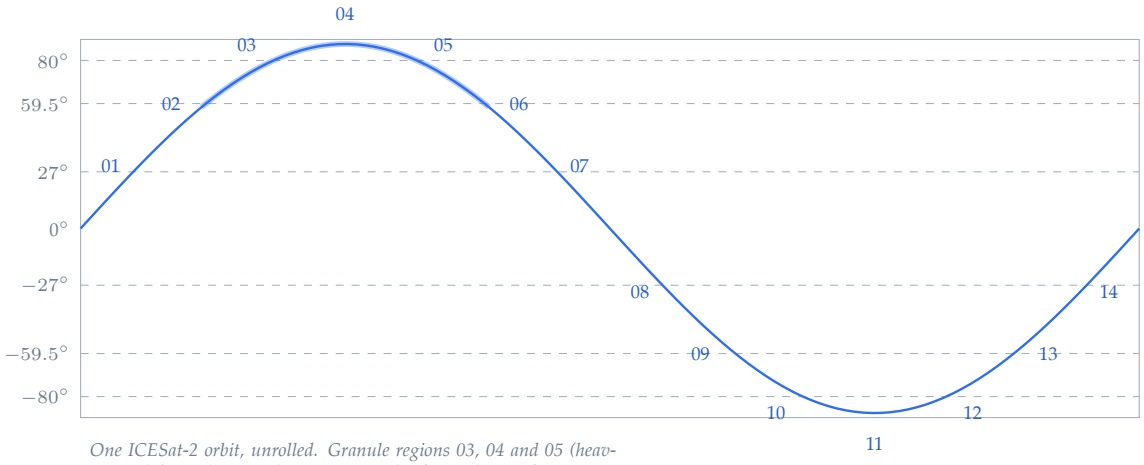


Figure 3: The granule is a product decision, not a geographic one. An ICESat-2 granule is one fourteenth of an orbit cut by latitude; a GLAS granule is a longer arc; an IceBridge granule is a few minutes of a flight. In every case the granule is hundreds to thousands of kilometres long and a study site needs a sliver of it.

2 Logical organisation: the granule and the product

The archive's unit is the *granule*: one file, catalogued once, with a footprint and a time range. What a granule contains is a product decision, and it differs by mission (Figure 3).

- An ICESat-2 orbit is cut into 14 *regions* by lines of latitude at 27°, 59.5° and 80° in each hemisphere, ascending and descending halves separately [10]. An ATL03 or ATL06 granule is one reference ground track (RGT), one cycle, one region: for Greenland, regions 03 and 05 (59.5–80°N, ascending and descending) and 04 (the arc above 80°N). Its name carries the start time, RGT, cycle, region, version and release: `ATL06_20181014050737_02380104_007_01.h5` is RGT 0238, cycle 01, region 04.
- A GLAH06 granule is a long orbit arc, pole to pole; the file name has no date, so the first shot's time is the only clock.
- An ILATM2 granule is a few minutes of one flight (about four minutes in the deployed index); a flight day yields dozens, a campaign thousands. Its name carries the date and the start time of the slice.
- A GEDI L2A granule is a quarter orbit, about fifteen minutes and ∼170 000 shots per beam, named by day-of-year, orbit and track.
- A traverse survey is one CSV file per monthly survey.

Inside a granule. The HDF5 products are trees of groups that hold datasets. ATL03 and ATL06 have one group per beam (`gt1l` … `gt3r`); which of the pair is the strong beam depends on the spacecraft's yaw orientation, a scalar stored once per file, and a yaw flip in progress leaves the question unanswerable for that granule. Within a beam, ATL06's `land_ice_segments` holds parallel arrays, one element per segment: latitude, longitude, height, time, a quality summary, and many more we do not read. ATL03's `heights` group holds one element per *photon*, and

its `geolocation` group holds one element per 20 m *segment*, with each segment recording the index of its first photon and its photon count; a photon's position is thus available both directly and, far more cheaply, through its segment. GLAH06 has no beams: one `Data_40HZ` group with one element per shot. GEDI has eight beam groups, each with per-shot arrays whose elevation is the lowest detected mode. The text products are one record per line: ILATM2 has eleven comma-separated fields per platelet (time of day, position, elevation, slopes, fit RMS, point counts, a distance, and the across-track *track* number, zero for nadir); the traverse file has twelve (position, antenna height, time, satellite geometry, and 95 % uncertainties).

Time is encoded five ways. ATLAS stores seconds since an epoch that is itself stored in the file, in GPS seconds since 1980, which differ from UTC by 18 leap seconds. GLAS stores seconds since J2000, noon on 2000-01-01. ILATM2 stores seconds of day, with the day in the file name. The traverse stores decimal hours with day-of-year and year in separate columns. GEDI's time we do not read: the date comes from its file name. A system that wants a temporal predicate has to normalise all five before it can compare anything, and the cheapest reliable clock is usually the file name.

Frames and heights. Heights are above an ellipsoid, but not the same one in the same frame. ATL06 is WGS84 in ITRF2014, with ocean tide and dynamic atmosphere corrections deliberately not applied. GLAH06 reports heights above the TOPEX/Poseidon ellipsoid in ITRF2008; the published conversion to WGS84 subtracts about 0.70 m. ILATM2's frame is whichever ITRF realisation the campaign used, read per granule from the header. The traverse is GRS80 in ITRF2020, and its surface height is the antenna height minus a sled geometry that changed in 2014. None of this concerns the index, which records where bytes are; all of it concerns the analyses that read the lake, which is why the lake stores each source's native height and the conversion lives in the source's descriptor (Chapter 21).

Quality. Each product has its own notion: ATL06's quality summary (zero is good); ATL03's per-photon signal confidence, a five-column array of which one column is the land-ice classification; GLAH06's elevation-use and saturation-correction flags; ILATM2's plane-fit RMS; GEDI's quality and degrade flags (which together keep 41 % of shots over Langtang); the traverse's height uncertainty. These are filters on *points*, applied after retrieval, and the only one the design asks the index to know about is the one that changes which bytes are needed: none of them do. Table 2 collects the per-collection facts the rest of the report relies on.

3 Physical organisation: how the bytes lie

Four of the six collections are HDF5 files; two are CSV text. The design needs only a small part of HDF5, but that part has to be exact, because the system will address bytes inside files it never opens again.

Chunks. An HDF5 dataset is stored in *chunks*: fixed-size pieces of the array, each compressed on its own (gzip, usually after a byte-shuffle filter

collection	granule is	positions	height	time	quality
ATL03	RGT×cycle×region	`lat_ph`, `lon_ph` per photon; segments carry a reference photon	`h_ph`	`delta_time` + SDP epoch (GPS)	`signal_conf_ph[:,3]` ≥ 3
ATL06	RGT×cycle×region	`latitude`, `longitude` per segment	`h_li`	`delta_time` + SDP epoch (GPS)	`atl06_quality_summary` $= 0$
GLAH06	orbit arc	`d_lat`, `d_lon` (0–360) per shot	`d_elev` + sat. corr. − T/P offset	`DS_UTCTime_40` (J2000)	`elev_use_flg` $= 0$, `sat_corr_flg` ≤ 2
GEDI L2A	quarter orbit	`lat/lon_lowestmode` per shot	`elev_lowestmode`	file name (day of year)	`quality_flag` $= 1$, `degrade_flag` $= 0$
ILATM2	minutes of flight	columns 2–3 per platelet (0–360)	column 4	seconds of day + file date	`track` $= 0$, RMS $<$ 50 cm
IS2TGPSSS	one survey	columns 1–2 per fix	antenna HAE − sled + track depth	decimal hour, DOY, year	`SDHGT_95` < 0.5 m

Table 2: The logical facts per collection. Everything in this table becomes a field of the collection descriptor in Part VIII; nothing in it is known to the shared pipeline.

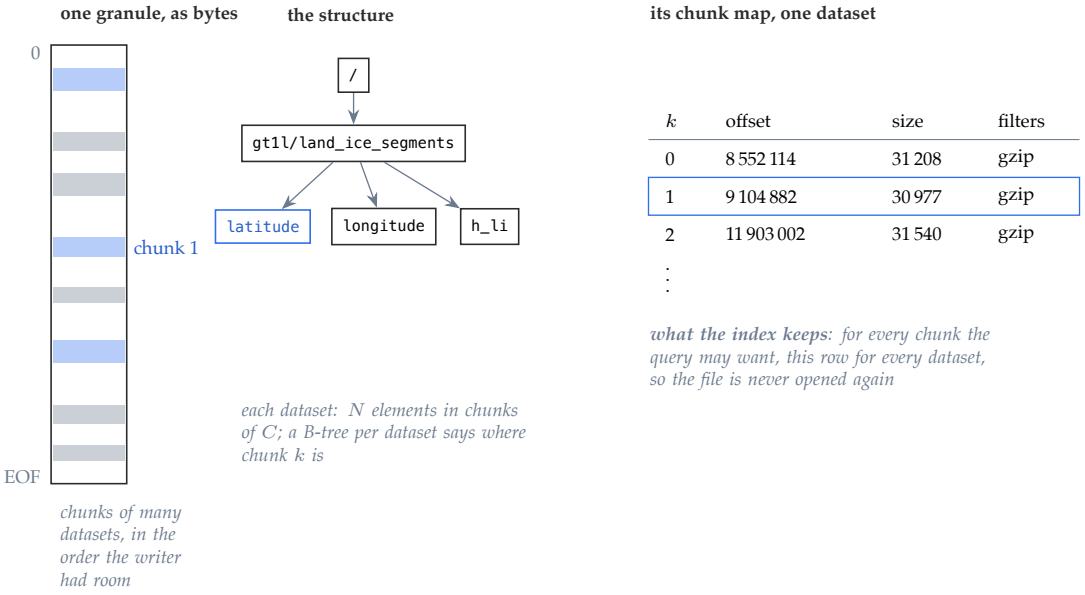


Figure 4: A granule's bytes, its structure, and the chunk map that connects them. The index is the chunk map, reorganised by where on the Earth each chunk's points fall.

that groups the bytes of each element's position to compress better) and written wherever the file had room at the time [15]. A B-tree per dataset maps chunk number to (offset, size) in the file. Reading element i means finding chunk i div C in that tree, fetching its bytes, decompressing them, and taking element i mod C. The chunk is therefore three things at once: the unit of compression, the unit of I/O, and the smallest thing a byte-range request can usefully ask for (Figure 4).

The chunk map (Figure 4) is the only thing the query needs from the file's structure. Walking the tree to obtain it is the expensive part of a build; Chapter 13 is about making that walk cheap. Decoding a chunk is the cheap, pure part: inflate, undo the shuffle, reinterpret as the dtype.[1]

Three layouts. The collections differ in how their datasets are chunked, and that difference decides what a row of the index can mean (Figure 5).

For the ICESat products and GLAS, every dataset of a group is chun-

[1] Decoding is validated byte-identical against the reference HDF5 library on real chunks; it needs no HDF5 code at query time.

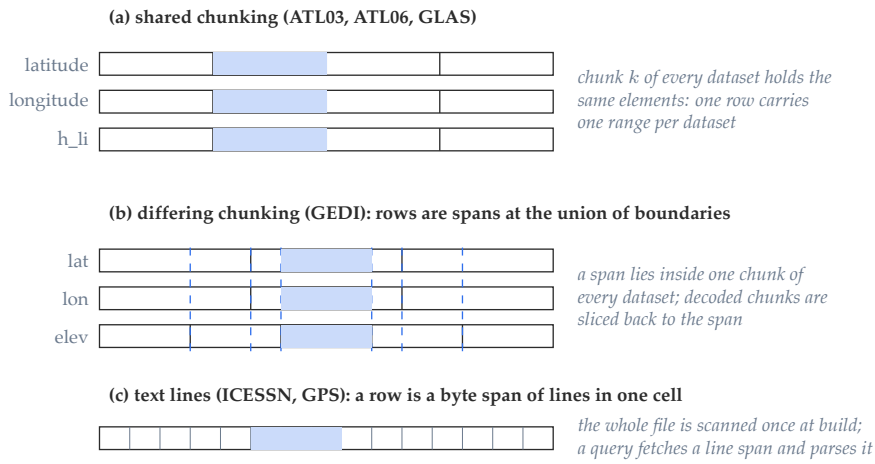


Figure 5: Three storage layouts, one index shape. In each case a row names a contiguous element interval and the byte ranges that hold it, so the query reads ranges and slices without knowing which layout produced the row.

collection	product	layout	position source	group structure
ATL03	ICESat-2 photons	shared chunks	segment reference photons	6 beams; strong/weak
ATL06	ICESat-2 land ice	shared chunks	per segment	6 beams; strong/weak
GLAS	ICESat GLAH06	shared chunks	per shot	one group
GEDI	GEDI L2A	spans	per shot	8 beams; full/coverage power
ICESSN	IceBridge ATM L2	text lines	per platelet line	none
GPS	Summit traverse	text lines	per fix line	none; sled geometry from a metadata file

Table 3: What differs between collections in layout. Each difference is a value or a small function in Part VIII.

ked identically (10,000 segments per chunk in ATL06, 100,000 photons in ATL03), so chunk k of the latitude dataset covers the same elements as chunk k of the height dataset, and one row can carry one byte range per dataset. GEDI chunks its datasets differently; the design cuts the element axis at the union of every dataset's chunk boundaries, so each resulting *span* lies inside exactly one chunk of every dataset and the row again carries one range per dataset, plus the offset of the span inside each chunk. The text collections have no chunks at all: the build scans the whole file once and a row is the byte span of the lines that fall in one cell.

ATL03's indirection. The photon product adds one twist that matters later. Photons are stored by time, and their positions are stored twice: every photon has its own latitude and longitude, and every 20 m *segment* has a reference photon and a count. A build cannot afford to read a billion photon positions, so it reads the segment positions and maps each segment's photon range (`ph_index_beg`, `segment_ph_cnt`) onto chunks. A chunk of 100,000 photons is thus located by the few hundred segments that cover it.

One shape, six sources. Table 3 is the whole of what differs in *layout*; everything else is shared. Read with Tables 2 and 4 it is, almost verbatim, the collection descriptor of Chapter 21.

collection	format	chunk (elements)	filters	granule size	indexed
ATL03	HDF5	100 000 photons, shared by the group	gzip (+ shuffle)	0.2–6.7 GB; Q 0.95 / 1.4 / 2.7 GB	48 838
ATL06	HDF5	10 000 segments, shared by the group	gzip (+ shuffle)	64–104 MB; Q 72 / 88 / 96 MB	33 484
GLAH06	HDF5	shared, one group	gzip	3–5 MB; Q 3.3 / 3.9 / 4.2 MB	4 799
GEDI L2A	HDF5	2 608 / 5 216 / 10 432 / 100 000 per dataset	gzip	∼100 MB	Langtang only
ILATM2	CSV	lines	none	0.1–0.5 MB	33 811
IS2TGPSSS	CSV	lines	none	0.1–1 MB	272

Table 4: The physical facts per collection. "Q" are size quartiles from the catalogue's size field over the Summit box (Langtang for GEDI). GEDI's four chunk lengths differ by 38× between datasets, which is why its index rows are spans rather than chunks.

Where the chunks lie. Nothing requires chunk k of one dataset to sit near chunk k of another, or near chunk $k + 1$ of itself, and the writers do not try. In the example ATL06 granule the five datasets' chunks for one 10 000-segment stretch of one beam are five ranges of 91 to 2 866 bytes that happen to lie within 8 577 bytes of each other, so one 8.6 kB request fetches the lot; in an ATL03 granule one chunk's five ranges can be 3.28 GB apart. The compressed size of a chunk also varies with content: a quality-flag chunk of 10 000 one-byte values gzips to 91 bytes, a latitude chunk of 10 000 doubles to 2.7 kB. The index must therefore record, per chunk, per dataset, an exact (offset, size), and the fetcher must be prepared to coalesce neighbours and to leave distant ranges as separate requests.

How much there is. Table 4 gives the chunk lengths and granule sizes measured on the deployed instance. Three numbers set the scale of everything that follows. An ATL06 granule over Greenland is 64–104 MB (quartiles 72 / 88 / 96 MB from the catalogue's own size field) and holds about five chunks of 10 000 segments per beam inside the region; its segments are 20 m apart, so one chunk is about 200 km of track. An ATL03 granule is 0.2–6.7 GB (quartiles 0.95 / 1.4 / 2.7 GB over Summit) and holds about a hundred chunks of 100 000 photons per beam. A GLAH06 granule is 3–5 MB and an ILATM2 granule 0.1–0.5 MB. The regional indexes on the deployed machine cover 33 484 ATL06 granules, 48 838 ATL03 granules, 4 799 GLAH06 arcs and 33 811 ILATM2 slices.

what exists today. the chunk map is read by `index._chunk_manifest` through h5py's `chunk_iter`; GEDI's span cutting is `index_gedi.chunk_spans`; the line parsers are `index_icessn._parse_fields` and `index_gpstruth`; sizes from `scripts/measure_search_modes.py` and `bench_build_io.py`.

region	bbox (W, S, E, N)	role
summit	$-39.5, 72.4, -37.5, 72.8$	cleanest interior; the ground-truth neighbourhood
summit_traverse	$-38.586, 72.579, -38.474, 72.648$	the 15 km GPS transect itself
egig_west_flank	$-45.0, 69.8, -43.0, 70.2$	default demo region; EGIG line, central-west flank
k_transect_upstream	$-48.5, 66.9, -46.5, 67.3$	upstream of the ablation zone
jakobshavn_margin	$-51.0, 69.0, -49.0, 69.4$	fast ice; the long-record showcase
ne_flank_upstream_negis	$-32.0, 75.5, -29.0, 76.0$	NE interior flank
langtang	$85.35, 28.20, 85.62, 28.38$	High Mountain Asia; the GEDI site

Table 5: The named areas the system is asked about. All but Langtang are on the Greenland ice sheet; the traverse box is 0.11° by 0.07°, the others 2–3° by 0.4–0.5°.

Part II

The problem

4 A canonical set of queries

Before designing anything we fix what the system is asked. The questions come from two places: the tools the deployed server exposes to an analyst or a language model, and the science questions the project set out to answer (the inter-mission elevation-change comparison, its feasibility over a candidate site, the long record at a glacier). Reduced to their shape, every one is a conjunction of four kinds of constraint.

Area. A named region; a rectangle in longitude and latitude; a polygon; a point and a radius; or a grid cell with a resolution. The deployed regions are 0.1–3° across (Table 5, Figure 6); a cell at the analysis resolution is about half a kilometre.

Time. A window of dates; the whole record; or a mission-specific bucket (a GLAS laser campaign, an ICESat-2 cycle, an IceBridge season). Every window is finally compared with a clock encoded five ways (Chapter 2).

Filters. Per-mission quality predicates (Table 2); which beams or beam groups (strong only, all six; full-power only); a confidence threshold for photons; a slope ceiling for GEDI.

Aggregation. None, the points themselves. Per-cell statistics: counts, passes, days, years, the height median and its spread. Co-located pairs between two sources with a difference and its robust spread. A per-cell time series with window medians and a trend.

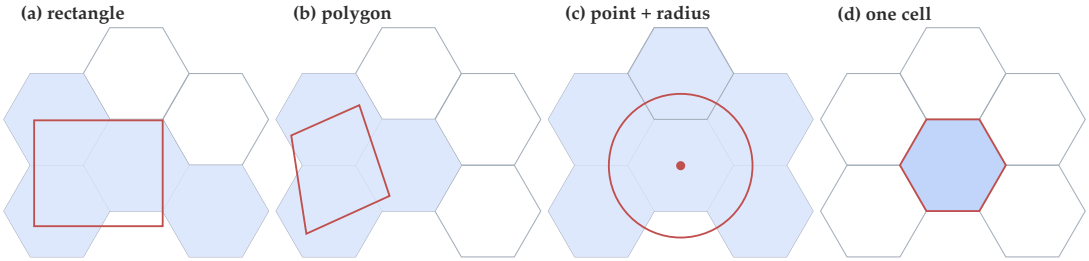
Six prototypes cover what is asked today (Table 6). They are written as specifications, not as procedures, because the design's first move will be to treat a query as a *value* that the system plans against rather than a sequence of calls.

Q1 Points. Every point of the selected sources inside an area over a window, with the per-source quality filter applied and the beam group chosen; returned as a table and drawn. This is the primitive; every other query is Q1 followed by arithmetic.

Q2 Coverage. For an area, per source: how many passes, distinct days and distinct years touch each cell at a coarse resolution, and

query	area	time	filters	aggregation
Q1 points	region / bbox / polygon	window or all	quality, beam group, confidence	none
Q2 coverage	point + radius, or bbox	all	none	passes, days, years per coarse cell
Q3 change	bbox or polygon	two windows	quality; both sources	co-located pairs, median, MAD
Q4 time series	bbox, then res-8 cells	yearly windows	quality; reference source	plane fit, window medians, trend
Q5 profile	thin polygon / line	window or all	quality, confidence (lower for water)	per pass
Q6 triage	point + radius	all	footprints	Q2 per source, with "never flew here"

Table 6: The six canonical queries as rows of the constraint algebra. Q1 is the primitive: every other row is Q1 over one or more sources followed by arithmetic on the points.



Shaded: the resolution-5 cells each area form implies. The cells over-cover the area by construction; a cell edge is about 10 km, so a rectangle 2° wide at 70° N (75 km) touches two or three cells across and the index answers for all of them.

Figure 6: Four area forms over the same ground, and the cells each implies. Whatever the form, the system reduces it to a set of cells and later re-applies the exact shape to the points.

whether a long record exists. Answered before any data are fetched; it decides whether a site is worth the fetch.

Q3 Elevation change between epochs. Points of two sources (GLAS and ATL06, say) inside an area, paired within 35 m, differenced after frame and epoch corrections, summarised as a median and a robust spread. Needs Q1 for two sources and a co-location.

Q4 Per-cell time series. For each cell at resolution 8 (about 530 m across) in an area: all points of all sources, binned into windows of a year, a fixed-effects surface removing the within-cell slope, window medians, and a least-squares trend with a confidence that ranks cells. Needs Q1 for every source and a lot of per-cell arithmetic.

Q5 Profile. Points along a line (a transect, a flight line, a lake shoreline) at full resolution, per pass, for a water-level or a slope question. A thin polygon in Q1's terms, with the aggregation per pass.

Q6 Triage. "Is site X feasible?" Q2 for every source plus the declared footprints: a source that never flew there must say so rather than report an empty index.

Two things about this set matter for the design. First, every query is spatial first: the time window, the filters and the aggregation are applied to points that an area selected. A system that can answer "which bytes hold points in this area" quickly has solved most of every query. Second, the same areas recur with small changes. An analyst nudges a box, adds a source, widens a window; a model asks Q2 and then Q4 for the same site. The work of fetching and decoding is worth keeping.

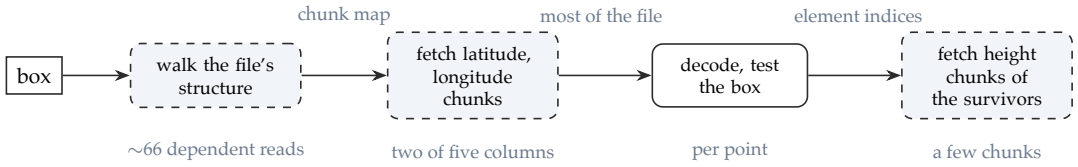
5 Search and retrieval today, and how they fail

The catalogue model. NASA's Common Metadata Repository (CMR) catalogues every granule with a spatial extent and a time range, and answers searches by collection, bounding box or polygon, and temporal range [16]. The spatial extent is whatever the data provider supplied: a polygon around the swath, a bounding rectangle, a line, or, for GLAS, only orbit parameters from which the catalogue back-computes a footprint. A client such as earthaccess [17] wraps the search and returns the granules' URLs; the data are then fetched whole, or opened remotely through a block cache that reads the file's structure on demand. The granule is the unit of discovery and the unit of retrieval. For gridded products this is adequate: a tile is where the catalogue says it is, and a tile is what you need. For along-track products it fails in seven distinct ways, and we measured each on the deployed instance against the index described later in this report (Table 7).

(a) The footprint is not the data. A granule's catalogued extent is a region the track passed through, not the set of places it measured. For ATL06 and ATL03 the extent is the polygon of a fourteenth of an orbit: 7–10 times the area of a $2^\circ \times 0.4^\circ$ study box and 850–1 000 times the area of the GPS transect box. A catalogue search for the box returns every granule whose polygon touches it. Of those, 10–14 % of ATL06 granules and up to 32 % of GLAS granules have no point inside the box's cells at all. The track clipped a corner of the polygon the box was in, or the instrument was off, or cloud took the surface. For IceBridge the earlier finding was starker: ATM's catalogued polygons over-claim the swath by up to about 990×, and indexing them as if they were data would have produced ∼16 % phantom cells [3]. For GLAS, only 13 of 30 granules the catalogue placed over one site passed within 2 km of it.

(b) The granule is far larger than the question. Even a true positive is almost entirely irrelevant. A 30 km box meets a few seconds of a granule that spans a quarter of the Earth. Downloading the catalogue's answer for one of our 2° boxes moves 45–50 GB of ATL06 and 1.0–1.4 TB of ATL03; the bytes that hold the box's points are 0.63–0.75 GB and 12–25 GB respectively, so whole-granule retrieval moves 55–95 times what is needed, and 475 times for the traverse box (Table 7). Opening the file remotely instead of downloading it does not escape this: the reader must still walk the file's structure (dozens of dependent small reads per file) and then read the position arrays to find out which elements are inside the box, which is most of the file's bytes for a three-column product.

(c) There is no map from geography to array index. This is the root of (b). In an along-track product the latitude is a dataset like any other, chunked and compressed. To learn which elements of a granule lie inside a box one must fetch and decode the latitude and longitude chunks, which is to say most of the granule, and only then can one ask for the heights of the elements that qualified: the question "where is 72.6°N in this file" can be answered only by reading the file (Figure 7). A grid product answers it with arithmetic. Nothing in the archive provides the equivalent for along-track data, and nothing in the file format could: the positions are measurements, not coordinates of a lattice.



Every step before the last is spent learning where the data are. The last step is the only one the question needed, and it is the smallest.

Figure 7: Finding a box's points in one along-track granule without an index. Position is data, so locating the data means reading it; the structure walk and the position fetch dominate, and both must be repeated for every granule and every query.

(d) The data are sparse and irregular. Across track, a site is sampled where tracks happened to cross it: for one 530 m cell the ICESat-2 record gives 855 populated cells where GLAS gives 112 over the same area, and IceBridge gives flight lines on 32 days in one cell and nothing in the next. In time, GLAS is 19 campaigns, GEDI paused for most of 2023, IceBridge flew in seasons. A coverage question (Q2) is therefore a real question, not a lookup, and a system that answers it by searching the catalogue returns the footprint's answer, which (a) showed is wrong by a third.

(e) Holes and fills. Points that exist in the file may carry no information: cloud returns, fill values (1.798×10^{308}, 3.4×10^{38}), saturated or degraded flags. On the deployed instance 4.1 % of the ATL06 cells a query selected turned out to name a chunk whose every point is fill. A system must keep the fact that it looked, or it will look again forever.

(f) The catalogue has its own pitfalls. Cloud-hosted and on-premises copies of the same granule are both listed, so an unfiltered search returns every file twice (2 898 hits against 1 449 for one area). A mistyped version returns zero granules silently, indistinguishable from an empty region. Polygon edges are great circles, so a 1.7° edge at 69°N bows 230 m and silently excluded three of 688 granules until edges were densified to 0.25°. A polygon of 725 vertices exceeds the request-URI limit. None of these is a defect to fix; they are the shape of a catalogue built for granule-level discovery, and a system that depends on it at query time inherits all of them.

(g) Time is never a predicate. With the clock encoded five ways and the granule as the unit, a time window can only be a granule-level filter on the catalogue's time range, which is the same over-claim as (a) in another dimension. Inside a granule, a window finer than its extent eliminates nothing a spatial predicate has not already eliminated: an ATL06 granule inside one res-5 cell is a few seconds long.

6 Requirements from first principles

Each failure in Chapter 5 says what a better system must record or do. We state the requirements once, numbered, and every mechanism in the rest of the report is introduced as the discharge of one of them (Figure 8).

R1 Search by where the points are, not where the track was. Failure (a). The system must hold, for every granule, a record of where its

area	source	CMR hits	GB	like-for-like	in cells	false pos.	need (MB)	ratio	footprint×
summit	ATL06	549	45.0	726	656	9.6 %	647	70	8.1
summit	ATL03	602	1 197	666	664	0.3 %	12 754	94	9.8
summit	GLAS	120	0.45	156	137	14.7 %	70	6.4	orbit
summit	ICESSN	195	0.03	271	251	7.4 %	25	1.3	0.0
egig_west_flank	ATL06	553	48.6	761	665	12.6 %	655	74	8.1
egig_west_flank	ATL03	560	1 040	656	600	8.5 %	12 072	86	8.5
egig_west_flank	GLAS	139	0.52	139	127	8.6 %	65	8.0	orbit
k_transect	ATL06	557	49.9	649	579	11.1 %	626	80	7.2
k_transect	ATL03	567	1 102	598	597	0.2 %	12 358	89	7.5
k_transect	GLAS	108	0.40	140	102	27.1 %	49	8.2	orbit
jakobshavn	ATL06	571	47.1	749	644	14.0 %	752	63	7.2
jakobshavn	ATL03	576	1 410	637	608	4.6 %	24 776	57	8.2
jakobshavn	ICESSN	504	0.14	882	775	12.1 %	102	1.3	0.0
summit_traverse	ATL06	109	9.0	220	150	31.8 %	105	86	852
summit_traverse	ATL03	116	234	175	144	17.7 %	493	475	1 009
summit_traverse	GLAS	15	0.06	31	21	32.3 %	11	5.3	orbit

Table 7: Catalogue search against the sub-granule index, measured 2026-10-19 on the deployed instance (`scripts/measure_search_modes.py`; full table in Appendix C). *CMR hits* and *GB* are the catalogue's answer for the box and its total granule size; *like-for-like* is the catalogue's answer for the outline of the box's cells, the same ground the index answers for; *in cells* is the index's count of granules with a point in those cells; *false pos.* is the share of like-for-like hits with no such point; *need* is the bytes the index would fetch; *ratio* is GB moved per GB needed; *footprint*× is the median catalogued footprint area over the box area ("orbit": GLAS records carry orbit parameters only, so no polygon exists to measure; IceBridge polygons are the thin swath, so their area is a fraction of the box's).

points fall, at a granularity fine enough to reject a granule that only brushes an area and coarse enough to be stored for fifty thousand granules. Such a record can be made only by reading the positions, so it is made *once*, at build time, never at query time.

R2 Retrieve by the storage unit. Failures (b) and (c). The smallest thing the archive can serve is a byte range, and the smallest useful byte range is a chunk. The record of **R1** must therefore be keyed by chunk and carry the chunk's (offset, size) for every dataset the query will want, so that a query goes from area to byte ranges without opening a file. The chunk map is read once, with the positions.

R3 A discrete, hierarchical, integer spatial key. The record of **R1** needs a key that a point maps to by arithmetic, that a query area maps to as a finite set, that is an integer (so files, tables and set operations stay cheap), that nests (so coverage can be stated compactly at one resolution and checked at another), and whose cells are of roughly equal area (so a resolution means the same thing everywhere). This is the definition of a discrete global grid system; Part III chooses one.

R4 Completeness must be provable. Failures (a), (e) and (f). Silence is indistinguishable from absence: an empty answer may mean no data, a granule not yet indexed, a search that returned zero by mistake, or a cell whose every point was fill. The system must record what ground each build covered, which granules it looked at, and which filter set each granule's record was made against, and a query must refuse an area the record does not provably cover.

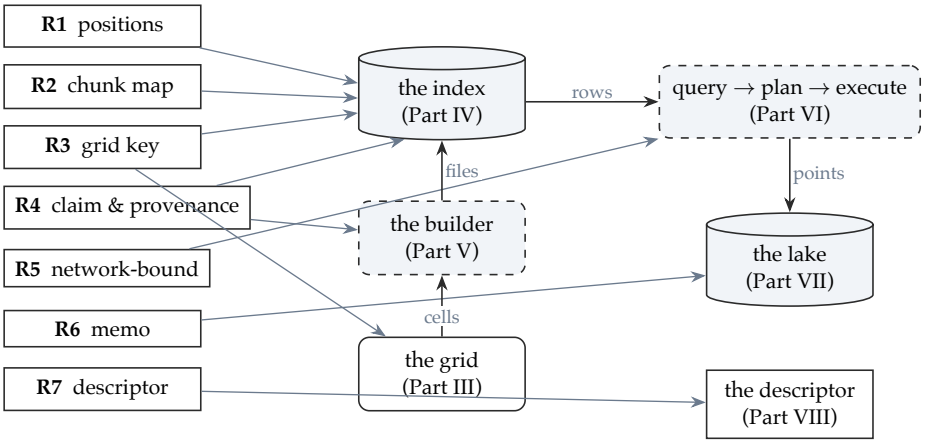


Figure 8: The requirements and the parts of the report that discharge them. The index is the centre: the grid keys it, the builder writes it, the query reads it, the lake memoises what the query decoded, and the descriptor is the one value that tells all of them which collection they are working on.

R5 A query's cost is bounded by the network. Failures (b) and (c). Once the index exists, the only irreducible cost of a query is fetching the bytes it names. Everything else (planning, deciding which points belong to which cell, decoding, caching) must cost less than the fetch, which means that work proportional to the number of points must be native and vectorised, or avoided by recording its answer in the index.

R6 Repeated nearby queries reuse decoded points. Chapter 4. Decoded points must be kept in a local store keyed by the same cells as the index, read before any fetch, and safe to write from several threads at once, so that the second query over a site costs only its arithmetic.

R7 A collection is data. Tables 2–3. Six sources differ in a dozen named ways and in nothing else. The pipeline must be written once, over a description of the collection, so that a seventh source is a description and not a re-implementation.

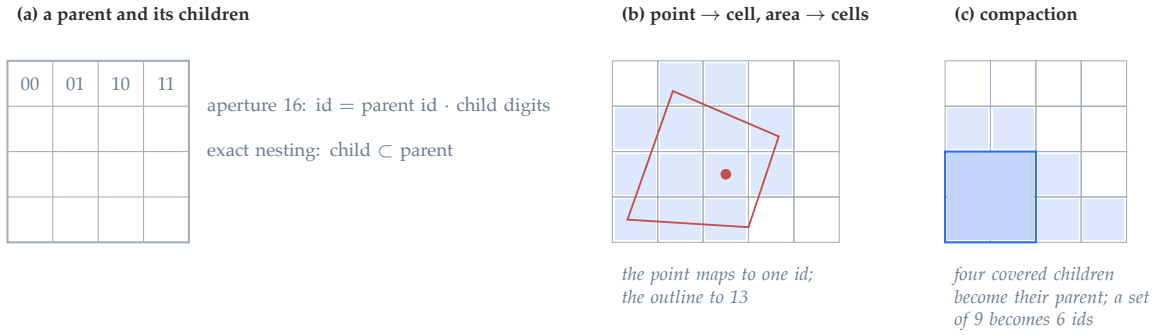


Figure 9: The three operations the index needs from a grid, shown on a square aperture-16 grid: the hierarchy encoded in the identifier, point and area assignment, and compaction of a set up the hierarchy. Each real system below realises them differently.

Part III

Discrete global grids

7 What a discrete global grid system is

R3 asks for a spatial key with five properties: a point maps to it by arithmetic, an area maps to a finite set of it, it is an integer, it nests across resolutions, and its cells have roughly equal area. A *discrete global grid system* (DGGS) is a family of partitions of the Earth's surface into cells, one partition per resolution, with a scheme that assigns every cell an identifier and relates cells across resolutions [18, 19]. The construction is always the same in outline (Figure 9): choose a polyhedron that approximates the sphere, project its faces onto the surface, tile each face with a regular pattern, and refine the pattern by a fixed *aperture* (the number of children per parent) to make the next resolution. The variations that matter to us are the polyhedron, the cell shape, the aperture, whether the refinement is exact (every child lies inside its parent) and whether the identifier is an integer that encodes the hierarchy.

What the index asks of these operations is specific, and worth stating as the tests a candidate grid must pass:

- **Point assignment** runs once per point at build time, for 10^9 photons and 10^8 segments. It must exist as a vectorised native kernel callable from Python over arrays; a scalar call per point is two orders of magnitude too slow.
- **Area to cells** ("polyfill") runs once per query and per build; it may be slow and must be exact in the sense of covering every cell the area overlaps, with a mode that excludes cells that merely share an edge.
- **Identifiers** are stored once per index row and once per lake file name, and compared in set operations by the million. They must be 64-bit integers, or packable to them.
- **Hierarchy** is used to compact a coverage claim and to test membership by walking ancestors. The identifier must yield the parent by

arithmetic. Exact nesting of *area* is a convenience the design must not rely on (Chapter 9).

- **Equal area** makes a resolution a statement about ground: "resolution 5" must mean about the same footprint at 60°N and at 80°N, or the chunk-to-cell ratio the index is tuned for drifts with latitude.
- **Convex cells** along a straight track make the run of a cell's points inside a chunk contiguous (Chapter 10); a grid with concave cells forfeits that shortcut.

8 Four systems

H3. Uber's H3 projects an icosahedron onto the sphere and tiles each face with hexagons; the twelve icosahedron vertices carry pentagons [20]. Resolution 0 has 122 base cells; each finer resolution has aperture 7, so a cell has one seventh the area of its parent and the finest resolution, 15, is under a square metre. Identifiers are 64-bit integers that encode the base cell and the sequence of child digits, so parent and children are bit operations. Because an aperture-7 hexagon tiling cannot be made of hexagons that tile their parent exactly, children are rotated by about 19.1° and straddle the parent's boundary: the hierarchy is exact in the identifier and approximate in area. Cells are convex, neighbours are uniform (six, except at pentagons), and area varies by about a factor of two across the globe at a given resolution. Vectorised kernels exist in Python through h3ronpy. H3 is what the system uses today (Figure 10).

S2. Google's S2 projects a cube onto the sphere and tiles each face with a quadtree [21]. A cell at level k is identified by a 3-bit face and k two-bit child digits packed into a 64-bit integer whose lowest set bit marks the level; the identifiers order cells along a Hilbert curve on each face, so consecutive identifiers are spatially close, and a parent's identifier is the midpoint of its children's range. Nesting is exact: a child lies inside its parent. Cells are quadrilaterals whose area varies by about a factor of two across a face because the gnomonic projection stretches the edges. Polyfill is a covering by a bounded number of cells of mixed level. Python bindings exist; a vectorised point kernel over arrays does not come with them (Figure 11).

HEALPix. The Hierarchical Equal Area isoLatitude Pixelisation divides the sphere into twelve base pixels of equal area, each refined by aperture 4 into pixels that remain exactly equal in area and whose centres lie on rings of constant latitude [22]. The equal-area and iso-latitude properties exist for spherical harmonic analysis of all-sky maps, which is its home. Nesting is exact and the "nested" numbering is hierarchical; identifiers are integers (64 bits suffice to resolution 29). Pixels are curvilinear quadrilaterals, not convex in the projected sense near the base-pixel corners. Vectorised kernels exist (healpy); polygon-to-pixel queries exist but are coarse (Figure 12).

rHEALPix. Gibb's rHEALPix rearranges the HEALPix projection so that the whole sphere maps onto a plane of square cells with exact aperture-9 (or 4) nesting and exact equal area, and extends it to an ellipsoid of revolution [23]. Cells are squares in the projection and

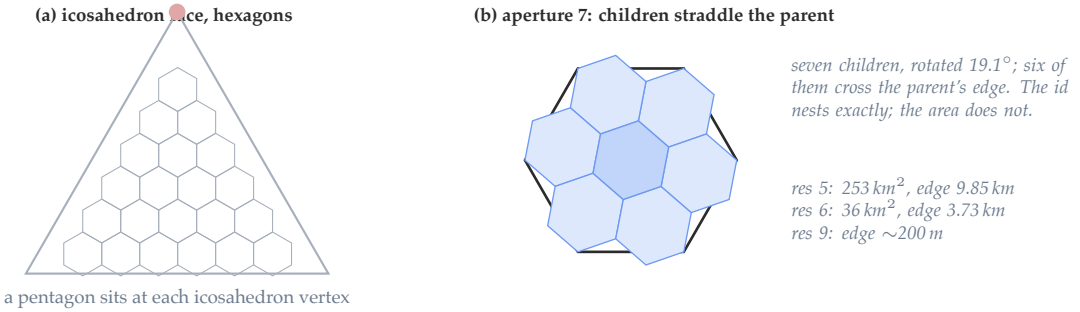


Figure 10: H3: hexagons on an icosahedron with aperture-7 refinement. Convex cells and uniform neighbours, 64-bit identifiers with the hierarchy in the bits, and a refinement that is exact in identifiers but not in area.

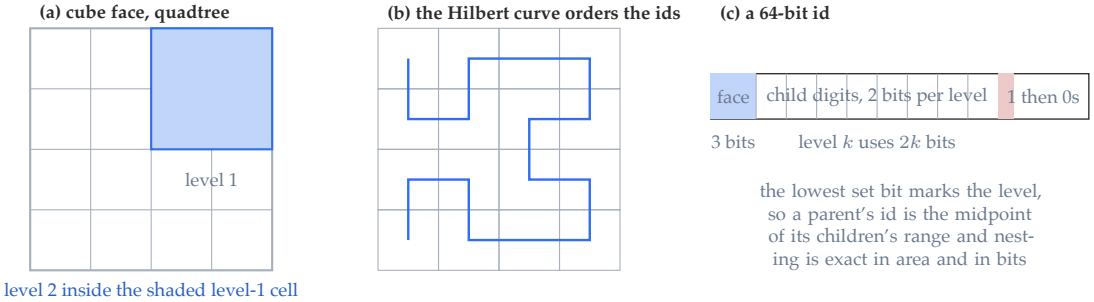


Figure 11: S2: a quadtree on each face of a cube, identifiers along a Hilbert curve. Exact nesting and spatial locality of identifiers; cells are quadrilaterals whose area varies across a face.

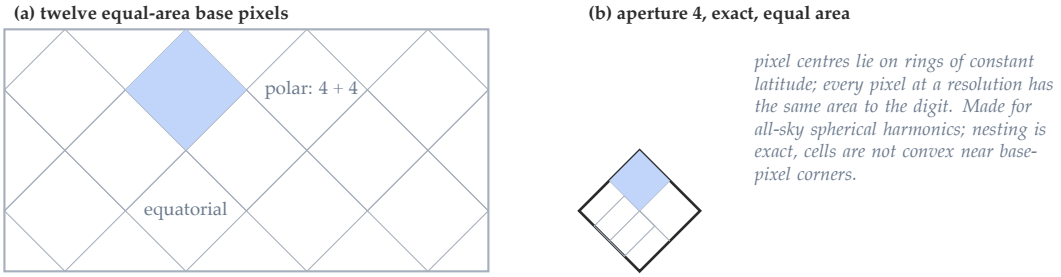


Figure 12: HEALPix: twelve base pixels refined by aperture 4 into exactly equal-area pixels on iso-latitude rings. Exact nesting, integer nested identifiers, vectorised kernels; cell shape is a curvilinear quadrilateral.

quadrilaterals on the ground, with the polar caps' squares distorted into triangles on the sphere. Identifiers are strings (a base cell letter and child digits); they can be packed into integers but no standard library does so. It is an OGC-conformant DGGS and its reference implementation is pure Python without a vectorised kernel (Figure 13).

The non-equal-area baseline. Geohash and the web-map quadkey partition the latitude–longitude rectangle, not the sphere, so a cell at 80°N is a sixth the ground area of a cell at the equator. They are cheap, hierarchical by string prefix, and everywhere. They fail the equal-area test badly enough for polar work that we do not consider them further.

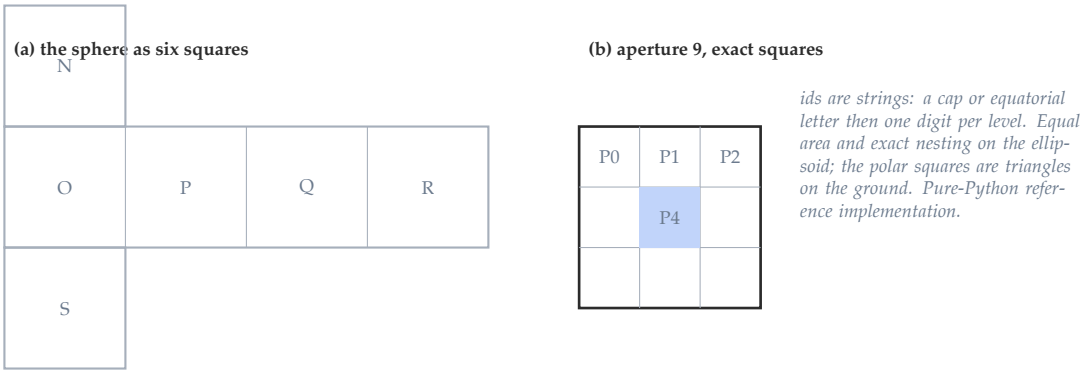


Figure 13: rHEALPix: HEALPix rearranged onto a plane of squares with aperture-9 refinement. Exact equal area and exact nesting; string identifiers; no vectorised kernel in the reference library.

property (test)	H3	S2	HEALPix	rHEALPix
polyhedron, cell shape	icosahedron, hexagon (12 pentagons)	cube, quadrilateral	12 base pixels, curvilinear quad	6 squares (plane), quad / triangle
aperture	7	4	4	9 (or 4)
identifier	64-bit int, hierarchy in bits	64-bit int, Hilbert order	int, nested or ring	string; packable
nesting in area	approximate (19.1° twist)	exact	exact	exact
equal area	$\sim 2\times$ variation	$\sim 2\times$ per face	exact	exact
convex cells	yes	yes	not at corners	yes (projected)
neighbours	6 (5 at pentagons)	4 edge + corners, mixed level	8, irregular at corners	8
polyfill with overlap mode	yes	covering, mixed level	coarse	yes
compaction	yes	yes (cell union)	by numbering	yes
vectorised point kernel (Python)	h3ronpy	no	healpy	no

Table 8: The four systems against the tests of Chapter 7. No system passes all of them; the deciding rows for a build over 10^9 points are the kernel and the identifier, and only H3 and HEALPix pass both.

9 The grid in this system

The system uses H3. The decision rests on three rows of Table 8: integer identifiers with the hierarchy in the bits (set operations, parent walks and compaction are arithmetic), a vectorised assignment kernel (the build's per-point cost), and convex cells (the contiguous-run shortcut of Chapter 10). HEALPix passes the first two and loses on the third and on tooling for polygon queries; S2 and rHEALPix lose on the kernel. What H3 gives up, exact nesting of area, the design is written not to need. Chapter 23 returns to the question as a parameter: the properties relied on are few enough to be named, and a grid that supplies them can replace H3 behind one value.

Resolutions. A resolution is chosen per collection so that a cell is roughly the ground footprint of one chunk: resolution 5 (253 km^2, edge 9.85 km) for ATL06, GLAS and IceBridge, whose chunks or arcs are hundreds of kilometres long; resolution 6 (36 km^2, edge 3.73 km) for ATL03, whose 100 000-photon chunks are about 70 km. Finer buys nothing on a scene-sized query and multiplies the index: the 2 016 842 resolution-5 cells of the globe become 14 117 882 at resolution 6. Analysis uses resolution 8 (about 530 m) for per-cell time series, and coverage claims are stated at resolution 9 (edge about 200 m) so that a claim follows a drawn shape closely; a claim is held under 60 000 cells by backing off one resolution at a time.

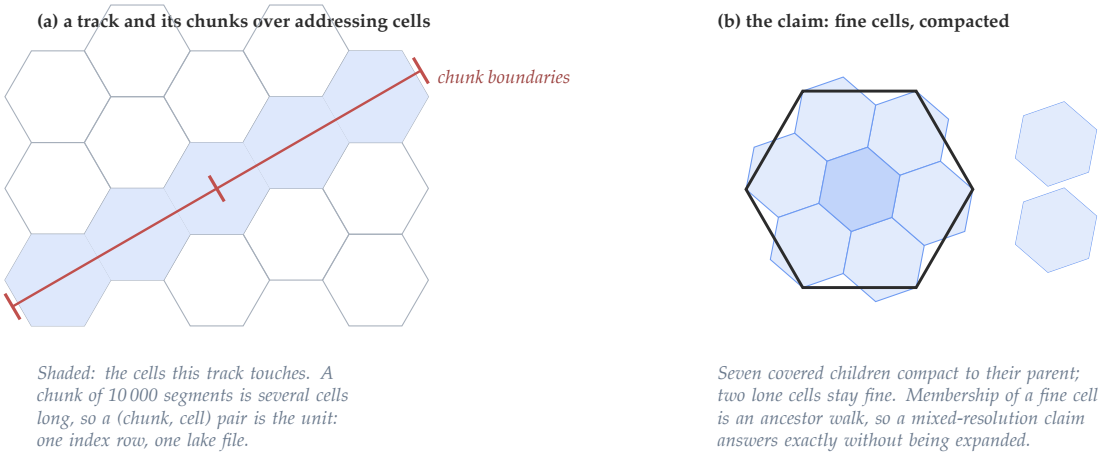


Figure 14: Cells in two of their three roles. Left, addressing: index rows and lake files are keyed by (chunk, cell). Right, claiming: the ground a build covered, stored compactly, consulted by every query and every enlarging build.

Three roles. A cell plays three roles, at two resolutions (Figure 14). *Addressing*: every index row carries the cell of its points at the collection's addressing resolution, and a query turns its area into the set of addressing cells it overlaps and asks the index for rows with those cells. *Partitioning*: the lake of decoded points is partitioned by the same cell, so a query reads exactly the cells it asked for and eviction has a unit. *Claiming*: a build records the ground it covered as a set of finer cells, compacted up the hierarchy; a later query checks that every fine cell of its area has an ancestor in the claim, and a later build over a larger area builds only the fine cells not yet claimed.

Two consequences shape everything downstream. First, a cell is coarse relative to a box, so the index over-selects by design and the query re-applies the exact box predicate to decoded points; the index is for finding bytes, never for answering the box. Second, the per-point assignment of a cell is the one operation the system performs *per point* on both sides, build and query, and it is the first thing to remove from the query side (Chapter 10).

what exists today. `planner.cells_for_bbox`, `coverage_cells`, `addressing_cells`, `search_polygon`; the claim in `index.write_build_manifest` and `unclaimed_cells`; the hot kernel is `planner._cells_vectorized` over h3ronpy.

Area to cells. A rectangle or polygon becomes its set of overlapping cells with the library's "overlap" containment, after the shape is shrunk by 2×10^{-6} degrees so that a cell which only shares an edge with the shape is not included. The cells over-cover the shape by up to one cell edge (10 km at resolution 5, 3.7 km at 6), so every query re-applies the exact shape to the points it decodes; the index finds bytes, it never answers the box. The cells also decide what the catalogue is asked at build time: their convex hull, densified to 0.25° edges so that great-circle bowing stays under five metres, is the search polygon, and it returns 688 granules where the exact outline would return 595.

Compaction and membership. A claim of 3 930 resolution-9 cells over a corridor compacts to 1 188 identifiers by replacing every complete set of seven children with their parent, recursively. Whether a fine cell is covered is then an ancestor walk, one parent operation per resolution step, with no expansion of the compacted set. Because H3's children straddle their parent, a compacted claim is a statement about *identifiers*, not about

ground: it means "every one of these cells' descendants at the claim resolution was built", which is exactly what the query asks, and nothing about the parent hexagon's outline is assumed.

Part IV

The index

10 The index as a relation

This chapter discharges **R1** and **R2**: a record, per granule, of where its points are, keyed by the storage unit and carrying the byte ranges that unit occupies. Logically the index is one relation, (granule, group, chunk, cell) $\rightarrow$ (byte range per dataset, chunk bounding box, run), and we derive it here; its physical form is the next chapter. Everything in Parts I–III can be assembled into the row the build writes. Take one granule, one group, one chunk k. The build has the positions of the chunk's elements (read once, from the position datasets or the segment table), assigns each its addressing cell, and so knows the set of cells the chunk touches. For each such cell it writes one row. The row has four parts:

- **Identity**: granule name, both URLs (HTTPS and `s3://`), the archive revision, the group (beam) and whether it is a strong beam, orbit numbers, the chunk index and the element interval the chunk covers.
- **The cell**, and the chunk's own bounding box in latitude and longitude. The box is a second, finer filter: a cell reaches kilometres outside a small query box, and the chunk's box prunes rows the cell let through.
- **The run**: where, inside the chunk, this cell's elements sit. Below.
- **Byte ranges**, one per dataset: offset, size, the filter pipeline, dtype, column count and the per-chunk filter mask. This is the chunk map of Figure 4, copied so the file need never be opened again.

Rows are few: a granule contributes about 18 rows for ATL06, 7 for ATL03, 25 for GLAS, 2 for ICESSN, because a chunk is long and a cell is large. The same chunk appears under every cell it touches, with identical byte ranges; the query deduplicates by (granule, group, chunk) before fetching.

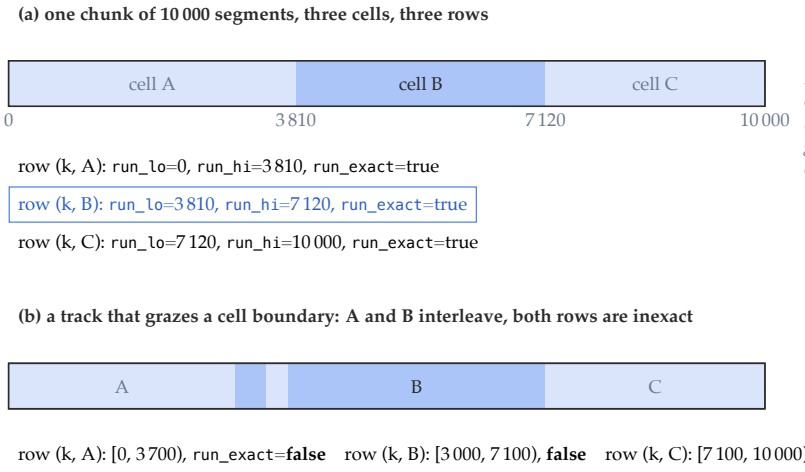
10.1 The run

The build already paid for the one per-point operation: it assigned every positioned element its cell. Without the run, the query would pay it again on every fetched chunk, because after decoding it has 10,000 segments or 100,000 photons and must know which of them belong to the cells it asked for. On the deployed machine that re-assignment was 59 million lookups and 24 to 31 thread-seconds of an 8-cell query.

The run records what the build learned (Figure 15). For a (chunk, cell) row:

$$[\texttt{run_lo},\ \texttt{run_hi}) = \big[\min i,\ \max i + 1\big) \quad \text{over this cell's elements } i,$$
$$\texttt{run_exact} = \big(\#\{i \in [\texttt{lo}, \texttt{hi}) \text{ positioned}\} = \#\{i \text{ in this cell}\}\big).$$

Exactness is decidable at build time because the build has every element's cell. It says: no element of another cell sits inside this interval. When every wanted cell's run is exact, the query labels the chunk's elements by



An exact run lets the query write the cell into a slice of the label array. An inexact run makes the query fall back to assigning every element of the chunk; it never guesses. Rows from an index built before the columns existed have no run and take the same fallback.

Figure 15: Cell runs. Each row carries the element interval of its cell inside the chunk and whether that interval is pure. The fast path is a slice; the slow path is the per-point assignment, kept alive by every inexact row.

slicing, in constant time per cell, and never calls the grid. A straight-ish track meets a convex cell in one contiguous stretch, so runs are exact unless cells interleave; on real ATL06 data 99.3 % of 9,422 runs over twelve granules are exact, and in an 8-cell ingest all 2,948 chunks were labelled from runs.

The two pure kernels are short enough to show whole, because they *are* the contract between build and query:

what exists today. `index.cell_runs`, `index.cells_from_runs`; the ATL06, GLAS and GEDI builders write `run_lo/run_hi/run_exact` (schema versions 3, 3, 2); their ingests slice. Counter `chunks_cells_from_runs` in the fetch stats.

```
def cell_runs(pos, cells) -> dict[int, tuple[int, int, bool]]:
    """{cell: (lo, hi, exact)} for one chunk; pos are within-chunk positions of
     its
    positioned points, cells their cells."""
    order = np.argsort(pos, kind="stable"); pos, cells = pos[order], cells[order
     ]
    out = {}
    for cell in np.unique(cells):
        idx = np.flatnonzero(cells == cell)
        lo, hi = int(pos[idx[0]]), int(pos[idx[-1]]) + 1
        inside = np.searchsorted(pos, hi) - np.searchsorted(pos, lo)   #
     positioned points in [lo, hi)
        out[int(cell)] = (lo, hi, inside == idx.size)
    return out

def cells_from_runs(n, runs, wanted) -> np.ndarray | None:
    """Labels for a chunk's n points, or None when the fast path does not apply.
     """
    if not runs: return None
    out = np.zeros(n, "u8")
    for cell in wanted:
        r = runs.get(int(cell))
        if r is None or not r[2]: return None      # missing or inexact: fall
     back
        out[r[0]:r[1]] = int(cell)
    return out
```

10.2 Photons: runs with edges

ATL03 complicates the run in one way, because a photon's cell is known to the build only through its segment. The build assigns a cell to each 20 m segment from its reference photon and expresses the run in photon index space: [`run_lo`, `run_hi`) is the union of the cell's segments' photon ranges

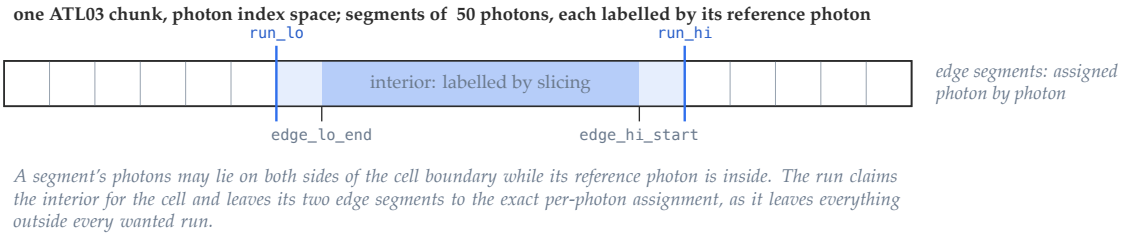


Figure 16: Photon runs. The interior of a cell's run is labelled by slicing; the two edge segments and all photons outside wanted runs are assigned individually, so the result equals assigning every photon.

inside the chunk. But a segment straddling the cell boundary has photons on both sides, and the index must not claim for a cell photons that belong to its neighbour. So the row records two more numbers, `edge_lo_end` and `edge_hi_start`: where the run's first segment ends and where its last begins (Figure 16). The query labels the interior by slicing and assigns the grid cell individually only to the photons of the two edge segments, and to photons outside every wanted run. The result is photon-exact, byte-identical to assigning all 100,000, at a few hundred lookups.

what exists today. `index.photon_cell_runs`, `labels_from_photon_runs`; ATL03 schema version 6; the planner's `_materialize` assigns only where `assign` is set and reports `photons_assigned`.

11 The index as files

This chapter discharges **R4**: the index must prove what it covers. Physically the relation of Chapter 10 is a set of Parquet files [24] plus three small side structures (Figure 17), and the proof lives in how they are named, stamped and folded.

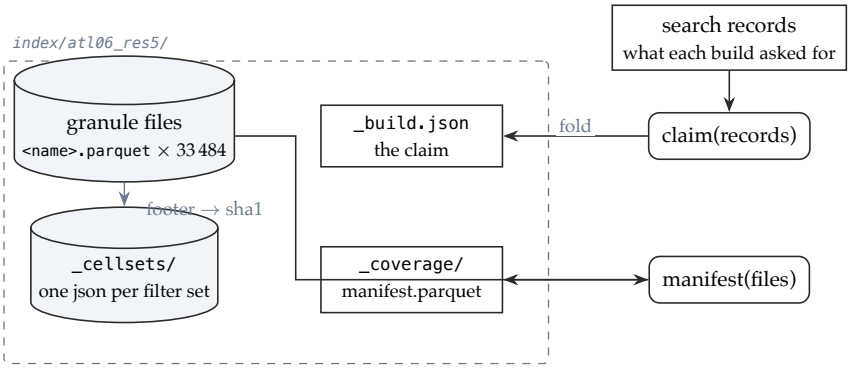
The rows of one granule are one Parquet file, named after the granule, under the collection's index directory. Writes are to a temporary name followed by a rename, so a reader never sees a torn file and an interrupted build leaves no corpse for the next run to trip on. The file is the unit of resumption: a build lists the directory, and a granule whose file exists with the current schema version is done.

That rule has a hole, and closing it is what the rest of this chapter is about. A build over a region asks for rows only in the cells of that region: a granule's pole-to-pole track would otherwise fill the index with cells no query ever asked for, each holding only the granules that happened to cross the region. So a granule's file is *relative to the cells the build asked for*, and a later build over a larger region cannot tell from the name whether the file holds rows for the added ground.

Provenance. Each file's footer therefore records the build's filter set: the compacted cell set the build asked for, stored once as a content-addressed sidecar (`_cellsets/<sha1>.json`) and referenced by its hash.[2] A granule *proves* a set of cells if its recorded set contains them. An enlarging build computes the fine cells not yet claimed and rebuilds exactly the granules that cannot prove them.

[2] The full compacted list was first stamped into every footer; at 49 KB it was three quarters of an ATL06 index file and 3.3 GB of a 7 GB store. The sidecar took the store to 4.7 GB.

The invariant is easy to state and was broken twice: the footer must record the *filter*, never the cells the granule happened to touch. Two builders had shadowed the filter variable with the per-point cell array. For GLAS the recorded set was touched ∩ filter, which was never wrong but gave every granule its own sidecar (4,220 for 4,799 files) and rebuilt everything on



The claim is derived*: a fold over the records of successful builds. The manifest is* derived*: a fold over the files. Neither is primary, both are cached because their derivation costs time, and both are rebuilt rather than repaired.*

Figure 17: One collection's index on disk. Granule files are primary and immutable; the claim and the manifest are folds over them. The provenance sidecars say which cells each file was built against.

every enlargement. For ATL03 it was the unfiltered touched set of the last beam, so a file whose rows had been dropped could prove ground it did not hold, and an enlarging build silently skipped it: the exact class of bug provenance exists to prevent.

what exists today. fixed 2026-10-17; the footer key is `aicesat_index_cellset_sha1`; `index.granule_proves` answers False for unknown provenance; `tests/test_build_provenance.py` pins both builders with a two-cell track and a one-cell filter.

The claim. When every granule of a build has landed, and only then, the build unions its fine cells into the collection's claim (`_build.json`): the compacted set of ground the index covers. Every query checks its area against the claim before doing anything else; an unclaimed area is an error naming the build command, not a slow success through whole-granule fallback. An empty catalogue search is recorded but claims nothing: silence from the catalogue is not evidence of empty ground.

The manifest. Coverage questions (which granules touch these cells, over which months) should not open tens of thousands of footers. The build ends by folding the index into one small manifest of distinct (cell, granule, month) rows (`_coverage/manifest.parquet`), rebuilt whenever the directory's modification time says a file landed. The query uses it to name the handful of files that touch its cells, so a plan reads a few files rather than a glob of 48,838.[3]

[3] Before the manifest the ATL03 plan for a 0.3° box scanned every footer: 36 s. Through the manifest, 0.66 s.

What the chapter establishes is a discipline for every derived thing in the system: the granule files are the primary data; the claim and the manifest are folds over them, cached because the fold is expensive, and never edited in place. Chapter 12 turns the build records into a value so the claim becomes a pure function of them.

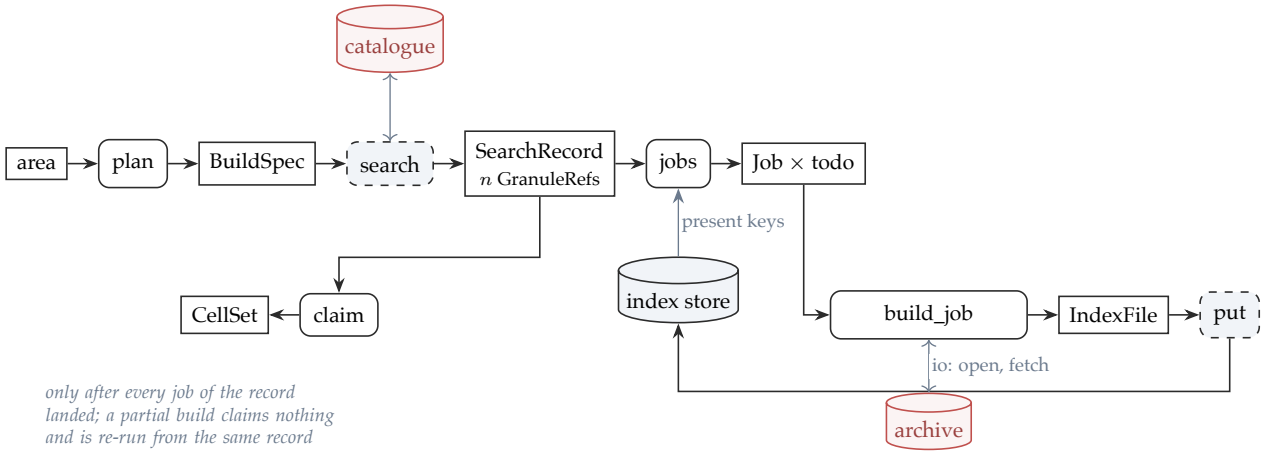


Figure 18: The build as stages over values. Rounded white boxes are pure; grey dashed boxes touch the catalogue, the archive or the disk. The record of a search is the primary value: jobs, outcomes and the claim derive from it.

Part V

A functional design for the index builder

12 Build as a pipeline of values

The builder is the effectful half of **R1** and **R2**: it reads every granule once, assigns its points to cells, records the chunk map, and writes the files of Chapter 11, proving (**R4**) what it covered. A build is one sequence, whatever the collection: decide the ground, find the granules, decide which still need building, build each, record what was built. Written as stages over values, with the effects named (Figure 18):

```
plan(area, coll)                 -> BuildSpec       pure: fine cells, addressing cells, search ring
search(spec)                     -> SearchRecord    EFFECT (catalogue); stored, append-only
jobs(spec, record, present)      -> tuple[Job]      pure: refs whose file is absent or cannot prove the ground
build_job(job, coll, cellset, io)-> IndexFile       pure given io = (open, fetch): rows + footer metadata
store.put(file)                  -> key             EFFECT: tmp + rename, write-once
claim(records)                   -> CellSet         pure fold; cached as _build.json
manifest(store)                  -> Table           pure fold; cached as _coverage/manifest.parquet
```

The values are small and frozen, and they are what crosses process boundaries:

```
@dataclass(frozen=True, slots=True)
class GranuleRef:    # what a worker needs: strings, not the catalogue's object
    name: str; url: str; s3url: str; revision: str; native_id: str = ""

@dataclass(frozen=True, slots=True)
class BuildSpec:     # the ground at both resolutions; the shape the catalogue
     is asked
    collection: str; res: int; schema: str
    fine_cells: tuple[int, ...]; addressing: tuple[int, ...]
    ring: tuple[tuple[float, float], ...]; window: tuple[str, str] | None

@dataclass(frozen=True, slots=True)
class SearchRecord:  # PRIMARY, append-only: what the catalogue answered, when
    spec: BuildSpec; at: str; granules: tuple[GranuleRef, ...]
```

```
@dataclass(frozen=True, slots=True)
class Job:           # one granule under one spec; its key names the index file
    collection: str; granule: GranuleRef; res: int; schema: str; cellset_sha:
     str; params_sha: str
    @property
    def key(self) -> str: ...   # sha1(name, revision, schema, res, cellset,
     params)[:20]

@dataclass(frozen=True, slots=True)
class Outcome:
    key: str; rows: int; error: str | None
    seconds: float; requests: int; bytes: int
```

Three design points follow from writing it this way.

The record is primary. Today the catalogue's answer is cached for a day as a pickle of the client's objects and the claim is a file mutated by each build. Storing `SearchRecord`s makes the claim a fold and makes replay possible (Chapter 24): a failed build is re-run from its record, not re-searched; what a build *would* do is computed from the record and the store's keys without touching the network.

The job key names the file. With content-addressed keys the resume set is a directory listing instead of a footer read per file (33,484 reads today, before the first job starts), and the provenance question of Chapter 11 is answered by the name: a file built under a different cell set has a different key. The GPS collection stamps a metadata hash into its rows for the same reason; that is the `params_sha` slot, empty for the other five.

Pure given io. `build_job` reads a remote file, so it is not pure in the strict sense; it is pure *given* two functions, `open` and `fetch`, passed in. In production they are the archive; in tests they are a synthetic HDF5 file in memory. Nothing else in the stage touches the world, which is what makes it testable per collection without a network and what lets a worker process run it with nothing but a `Job`.

what exists today. `build.GranuleRef` (four strings plus a compatibility shim for the builders), `build.BuildCollection` (the per-collection parameters), `build.run_build` (the sequence, once, for ATL06, GEDI, GLAS, ICESSN and GPS) over `pool.run_pool`. Still proposed: `SearchRecord` stored, content-addressed keys, the claim as a fold.

13 Reading one granule

Failure (c) of Chapter 5 does not disappear because the read happens once: the build still has to find the chunk map and the positions inside a file whose structure is a tree of dependent reads. This chapter is about paying that cost once as cheaply as possible.

The build's cost is in `build_job`, and inside it almost entirely in reading the file's structure. Before any position is read, the HDF5 library walks the superblock, the group tree and the chunk B-trees: for an ATL06 granule about 66 small reads, each a dependent round trip, because the next address is learned from the previous read. Through a 1 MiB block cache over the object store that walk took about 2 s of a 3.3 s granule; the position fetch 0.5 s; the actual computation 0.3 s. Workers scaled the wall time until the machine ran out of threads to wait on, and then not at all (Table 9).

A dependent walk cannot be parallelised, but it can be made local. The granule sizes decide how (Figure 19):

- **Small granules (ATL06, GLAS, 2 to 128 MB): read the whole object once.** The first 8 MiB span is requested alone and the object's size read from the response's `Content-Range`, with no `HEAD` and no

workers	wall (s)	granules/s	s per granule per worker	CPU busy
4	81.0	1.23	3.24	20 %
8	41.9	2.39	3.35	44 %
16	26.2	3.81	4.20	80 %
32	26.2	3.81	8.39	92 %

Table 9: ATL06, 100 granules, 4 vCPU, before the whole-object read: latency-bound to 16 workers, then flat. The per-granule time is the sum of serial round trips.

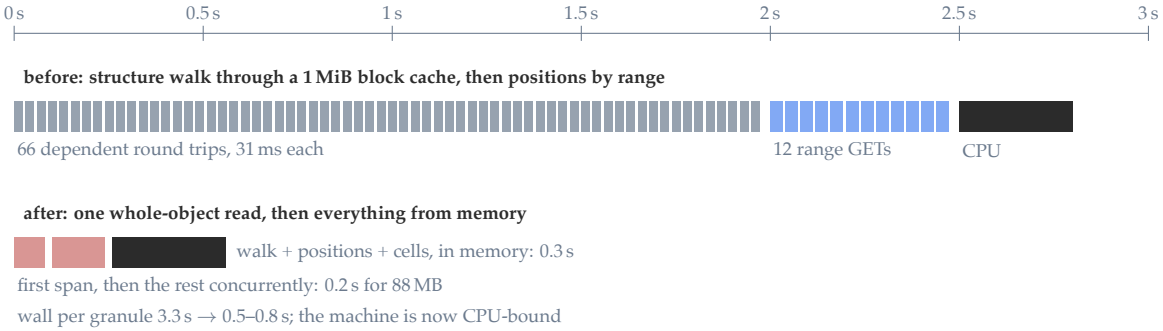


Figure 19: One ATL06 granule's build, before and after, on the same time axis. The serial walk disappears into a buffer; what remains is computation, so adding workers past the core count buys nothing and the lever becomes cores.

extra round trip; the remaining 8 MiB spans go out in one concurrent submission; the structure walk then runs over an in-memory buffer at memory speed, and the position chunks are sliced from the same buffer. Objects over a cap (256 MiB, since the per-worker peak is two to three times the object) fall back to the block cache having spent one span.

- **Large granules (ATL03, 0.2 to 6.7 GB): read bigger blocks.** The structure of an ATL03 file is clustered, so a 4 MiB block satisfies the walk in 7 reads instead of 114 while moving 29 MB instead of 120. Larger blocks read more for nothing (16 MiB: 4 reads, 67 MB, no faster).

Both cases go through one door. A `GranuleSource` is what a builder opens: `open()` returns the object the HDF5 library parses (a memory buffer, or the block-cached remote file), `fetch(ranges)` returns byte ranges from the *same* bytes (sliced from the buffer, or range GETs), and `stats` counts what was moved. Every dataset a builder reads goes through one function, `read_via_chunks`, which takes the dataset's chunk map and asks the source for the chunks, decoding them with the same decoder the query uses. The builder therefore does not know which path it is on, and a test can hand it a buffer of a synthetic file.

The measurements (Table 10) say the build is no longer waiting on the network. What is left is computation in the worker, so the next lever is cores, not workers.

what exists today. `access.read_whole_object`, `GranuleSource`, `open_granule`; `index.read_via_chunks`; `index.ATL03_BLOCK_SIZE = 4 MiB`. ATL06 and GLAS opt in to the whole object; GEDI stays on the block cache until its sizes are measured.

14 The build at runtime

The structural view says what is computed; this chapter says by whom. A build is one parent process and a pool of worker processes (Figure 20). The parent does everything that is cheap and must happen once: plans

collection	configuration	before	after	note
ATL06, 100 granules, 8 workers	whole object	44.7 s	21.3 s	CPU 46 % → 94 %
ATL06, 300 granules, 6/8/12 workers	whole object		5.4 gran/s	flat in workers, CPU 92–98 %
ATL06, full regional rebuild (33 484)	whole object	3.9 h	62 min	9.1 gran/s sustained
GLAS, 35 granules, 8 workers	whole object	7.4 s	4.1 s	
GLAS, full rebuild (4 799)	whole object	10 min	3.5 min	
ATL03, 20 granules, 8 workers	4 MiB blocks	23.4 s	11.1 s	114 → 7 structure reads
ATL03, full regional rebuild (48 838)	4 MiB, 16 workers	12 h	≈7.3 h	1.88 gran/s, CPU-bound

Table 10: Build measurements on the deployed instance (m6i.xlarge, 4 vCPU, in-region), 2026-10-17.

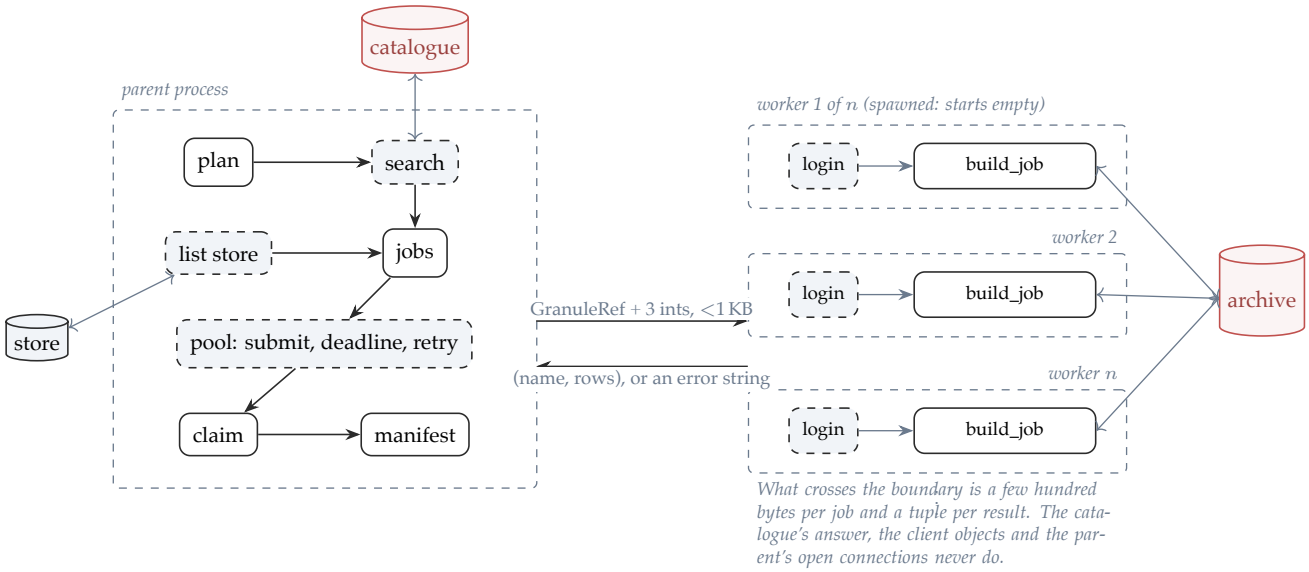


Figure 20: The build at runtime. One parent, n spawned workers, each holding its own archive client. The arrows carry values small enough to pickle without thought.

the ground, logs in, searches the catalogue, lists the store, decides the jobs, and at the end claims and folds the manifest. A worker does one thing: `build_job` for the job it is handed, with its own login and its own archive client.

Spawn, not fork. A forked worker begins as a copy of its parent. For a regional ATL03 build the parent holds the catalogue's answer, several gigabytes of client objects, and copy-on-write un-shares those pages worker by worker until memory approaches $(n+1)$ times the parent; a 16-worker ATL06 build was killed that way. A spawned worker begins empty and receives only its job. The price is a second of imports and a login per worker, paid once per build.

One policy for the pool. Five builders once owned five pools and disagreed on everything that matters. The one pool now has one policy: submit every job; consume the futures *in submission order* with a per-future timeout computed from a monotonic clock (a laptop that sleeps must not report a healthy build as timed out), so a progress callback sees jobs in a stable order; treat every worker exception as retryable, because the real cause of a transient archive error is destroyed in transit when the error object cannot be pickled back; run a second attempt for the failures in a fresh pool; then cancel what is left (Figure 21).

what exists today. `pool.run_pool`, called with exactly `max_workers` and `mp_context`; `build.run_build` claims only when every job of the attempt landed and otherwise logs that it is not claiming.

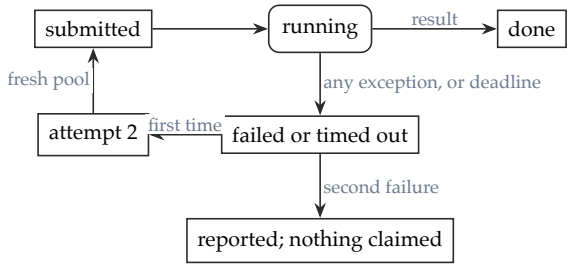


Figure 21: The life of one job in the pool. The claim at the end of the build requires every job to reach *done*; a single job in the bottom state withholds it, and the next run resumes from the files that landed.

How many workers. After Chapter 13 an ATL06 granule costs about 0.7 core-seconds, so on four vCPUs the knee is at six to eight workers and sixteen buys nothing but memory. The text collections and ATL03 are still latency-bound (a whole-file GET, or a large structure walk) and take sixteen. Memory for sixteen spawned workers with whole-object buffers peaked at 3.4 GB on a 15 GB machine; two collections are never built at once.

what exists today. `BuildCollection.default_workers`: 8 for the HDF5 builders, 16 for ATL03, ICESSN and GPS; GEDI's per-granule timeout is 600 s because a granule is a quarter orbit.

Part VI

From query specification to plan to execution

15 The query specification

Part VI discharges **R5**. Chapter 4 wrote the canonical queries as conjunctions of area, time, filters and aggregation; the design's first move is to make that conjunction a value. A `Query` names the collection, the area (a rectangle and, optionally, the polygon inside it), the window, the beam group policy, and two flags (force a re-fetch; clip to cells rather than to the rectangle). It carries no derived state: its cells, its catalogue of index rows and its split into what the lake has and what must be fetched are all functions of it and of the stores, computed by the planner of the next chapter. Normalisation happens at the door: a region name becomes its rectangle, a point and radius become a cell or a rectangle, a window becomes two `YYYY-MM-DD` strings compared against the file-name clock. The per-mission filters of Table 2 are not part of the query at all; they are the collection's, applied to decoded points, and the query can only choose among the few policies the collection offers (which beam group; a confidence threshold for photons).

16 The plan

A query begins as a value: the area (a box, or a polygon inside it), the time window, the collection, and a few flags (which beam groups, whether to force a re-fetch, whether to clip to cells). Turning it into points has a planning half that touches only local stores and a fetching half that touches the archive. The planning half is worth isolating because it is cheap, deterministic given the stores, and what every inspection tool (a dry run, a byte-range map, a progress denominator) wants to see.

```
cells(query, grid)                -> tuple[int]       pure: addressing cells the area overlaps
scan(query, coll, store)          -> rows             EFFECT (DuckDB over the manifest-named index files)
have(coll, granules, lake)        -> set[ChunkKey]    EFFECT (the lake's key table): what is already decoded
plan_query(query, coll, rows, have) -> QueryPlan      pure: todo = chunks with any wanted cell not in have,
                                                            completed to whole beam groups, grouped by URL
render(plan)                      -> dict             pure: what the chunk map and the progress bar show
```

The scan asks the index for every row whose cell is wanted, restricted to the beam group and window, reading only the files the manifest names. Rows arrive one per (granule, group, chunk, cell), each carrying its run. The plan groups them by chunk: a chunk's wanted cells, its runs, one representative row for its byte ranges. Against the lake's key table it splits chunks into *have* (every wanted cell already decoded) and *todo* (Figure 22). Two refinements are forced by later chapters and are decided here, not downstream:

- **A beam group is fetched whole.** If any beam of a group needs chunk k, the group's other beams' chunk k join the todo set. Chapter 18 explains why the lake's file unit makes this necessary.
- **Writes are narrowed to the wanted cells.** A chunk is far longer than a box and decodes points in cells the query never asked for.

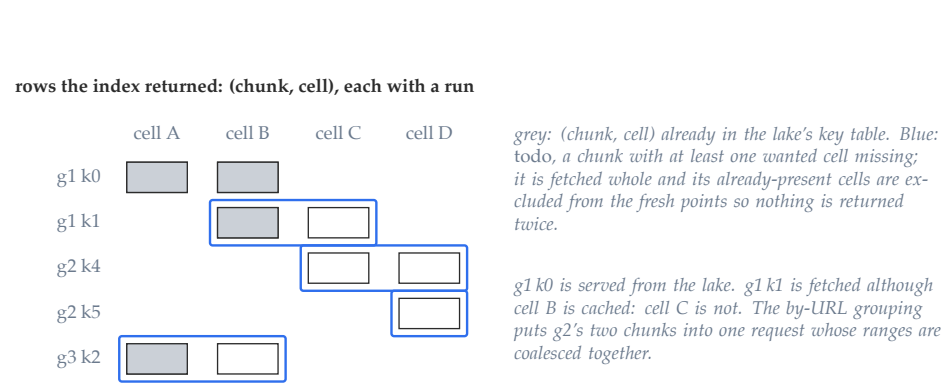


Figure 22: The plan as a set picture. The lake's key table decides *have*; a chunk goes to *todo* if any wanted cell of it is missing; todo is grouped by granule URL so one request per granule carries every range.

The plan records `want_only`, the cells the lake may write; the rest of the strip is discarded (Chapter 20 has the measurement that decided this).

The plan is a value and should be serialisable as one: the query, the rows as an Arrow table, the have set and the todo list. Chapter 24 uses that to replay an ingest without the planning half.

what exists today. `index_atl06.plan_bbox` returns this dictionary without touching the network (keys `want_cells`, `rows`, `chunk_cells`, `chunk_runs`, `chunk_row`, `have`, `todo`, `n_lake`, `by_url`, `want_only`); `_whole_groups` completes beam groups; the ATL03 planner does the same inline in `planner._ensure`.

```
@dataclass(frozen=True, slots=True)
class Query:
    collection: str; bbox: tuple[float, float, float, float]; polygon: tuple |
     None
    window: tuple[str, str] | None; res: int
    groups: tuple[str, ...] = ("strong",)       # beam groups wanted
    force: bool = False; clip_cells: bool = False

ChunkKey = tuple[str, str, int]                 # (granule, group member, chunk
     index)

@dataclass(frozen=True)
class QueryPlan:
    query: Query
    want_cells: tuple[int, ...]
    rows: pa.Table                              # one row per (chunk, cell),
     with its run
    have: frozenset[tuple[str, str, int, int]]  # (granule, beam, chunk, cell)
     in the lake
    todo: tuple[ChunkKey, ...]                  # whole beam groups
    by_url: Mapping[str, tuple[int, ...]]       # url -> row indices
    want_only: tuple[int, ...] | None
```

17 Executing the plan: one chunk, step by step

Execution is the same for every todo granule and, inside it, for every chunk. This chapter follows one chunk through it and states, for each step, what the work is proportional to (Figure 23). The distinction between per-point, per-chunk and per-granule work is the whole performance story of the query side: anything per point must be native and vectorised, anything per chunk may be Python, anything per granule may be a network round trip.

Coalesce and fetch. The granule's todo rows name $5 \times m$ byte ranges. Adjacent ranges separated by less than a gap are merged into one span,

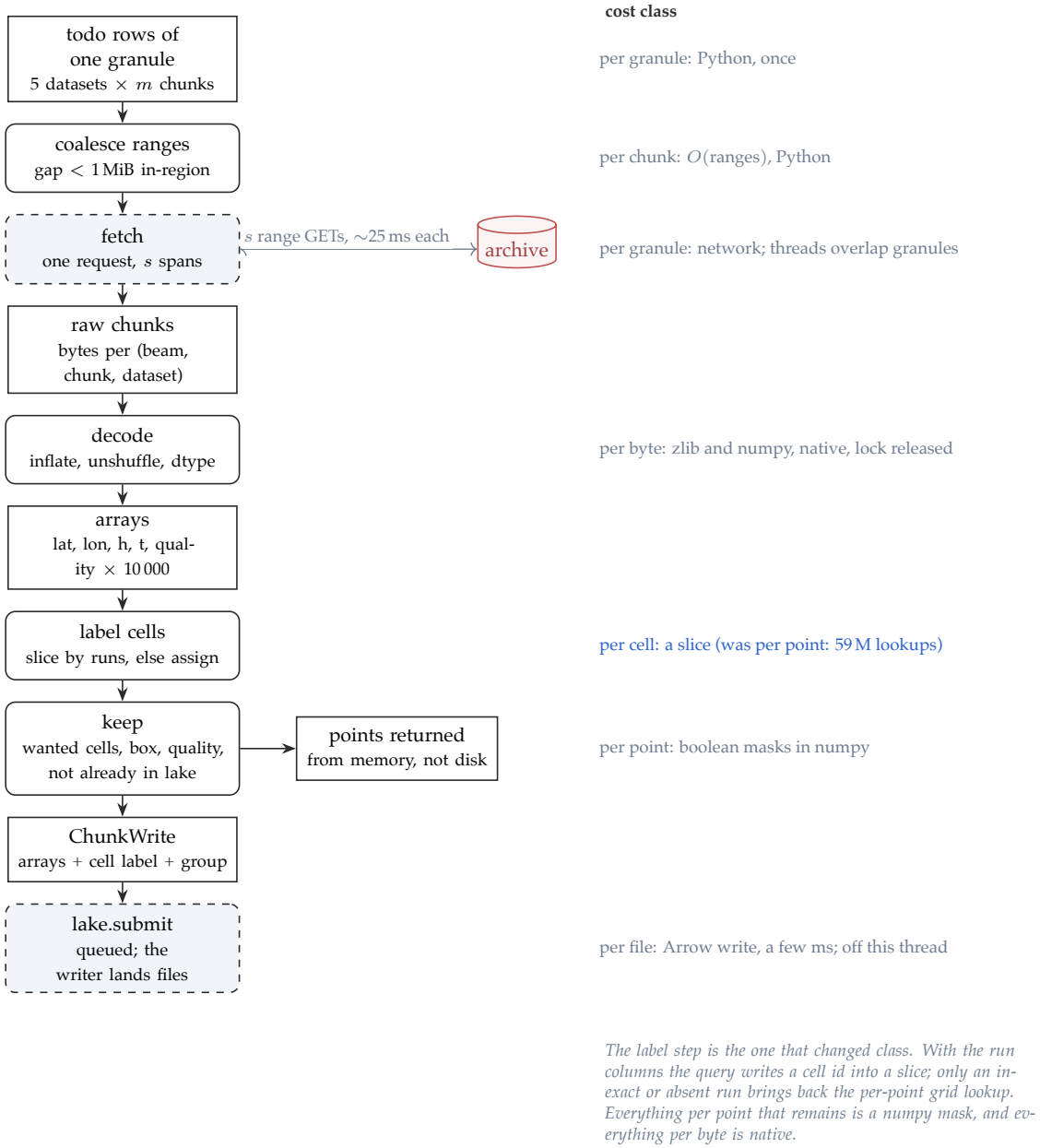


Figure 23: One chunk through the executor, with the cost class of each step. The returned points come from memory; the lake write is housekeeping for later queries and runs on other threads.

because a request costs a round trip and bytes in the gap cost only bandwidth; the gap is tuned by measurement to where the two balance (about 128 KiB from outside the region, 1 MiB inside). One request per granule carries every span; granules are fetched on a thread pool sized for latency, not cores, because a thread waiting on a GET uses no CPU.

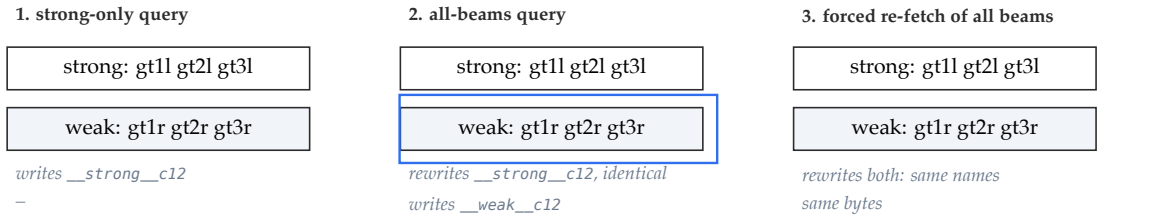
Decode. Each raw chunk is inflated, unshuffled and reinterpreted as its dtype. The three operations release the interpreter lock and run at memory speed. The decoder is the same function the build used to read positions, so a byte-identity test on one side covers the other.

Label. This is where Chapter 10 pays. For each wanted cell of the chunk the run is written into a slice of a label array; a cell absent from the runs or with an inexact run sends the whole chunk to the per-point assignment. ATL03 does the same with its edge columns and assigns only edge photons and outsiders.

Keep. One predicate, applied once to the chunk's arrays as boolean masks: the label is a wanted cell; the point lies inside the box (unless clipping to cells); the quality flag passes; and the cell is not one the lake already returned for this chunk, so a partially cached chunk never contributes a cell twice. The kept points are the answer; they are returned from memory and streamed to the caller granule by granule as each fetch lands.

Hand off. The chunk's full arrays, the label and the beam group go into a `ChunkWrite` value and onto the lake's queue. The executor does not write files, wait for them or know their names. One predicate, one label, one value crossing to the writer: the five copies of this loop that exist today differ only in the per-collection decode and quality functions, which are the slots of Chapter 21.

what exists today. `index_atl06.fetch_bbox` with its inner `_ingest_granule` and `_keep`; `access.coalesce`, `decode_chunk`, `unshuffle`, `map_as_completed`; the ATL03 path in `planner._process_group`, `_decode_photons`, `_materialize`. Five copies of the loop remain (ATL06, GLAS, GEDI, ICESSN, GPS) plus the ATL03 variant.



Because the file is a full overwrite of a deterministic name, the planner must fetch a group whole: if one beam of a group were re-fetched alone, its file would be rewritten with that beam's points only while its siblings stayed marked as present. Chapter 16's whole-group rule is this file's correctness condition.

Figure 24: Idempotence across the two beam sets the planner can ask for. Each group's file has one name and one content; nothing merges, nothing locks, nothing lists a directory.

Part VII

The retrieval engine and the lake

18 The lake as a memo

This part discharges **R6**.

The lake is a cache of decoded points, partitioned by cell, read before any fetch and written after. Losing it costs time, never truth: anything missing is re-fetched. Its design is dominated by one requirement, idempotence under concurrency, and one measurement, the cost of a file.

Layout. One Parquet file per (cell, granule, chunk, beam group), under a directory per cell:

```
lake/mission=ATL06/h3_cell=599686042/<granule stem>__strong__c12.parquet
lake/mission=ATL06/h3_cell=599686042/<granule stem>__weak__c12.parquet
lake/mission=GEDI/h3_cell=.../<stem>__full__c3.parquet        full / coverage power
lake/mission=ICESSN/h3_cell=.../<stem>__na__c0.parquet        beamless collections
```

Every point goes to the file of its *own* cell, so a later query for a different sub-box of the same cell re-filters correctly; a partial cell is impossible by construction. The cell directory is the unit of reads (a query opens only the directories it asked for) and of eviction. A DuckDB table beside the lake records which (granule, beam, chunk, cell) keys have landed; it is the *have* of Chapter 16.

Why a beam group. The file count is the cost. Six files per (cell, chunk), one per beam, were 6,897 files for 2,960 chunks in one 8-cell ingest at 3 to 6 ms each, and merging chunks of one beam was measured to be a no-op because a scene box spans about one chunk per beam. Merging *beams* cuts the count, but merging all six is unsafe: the planner asks for the strong beams by default and for all six from the extraction API, so one name per (cell, chunk) would be overwritten by whichever beam set came last. The group is the unit the planner requests (`strong`, `weak`; `full`, `coverage`), so a group's file is always a full overwrite of a deterministic name with the same content, whichever request produced it (Figure 24). Six files became two: 3,225 for the same ingest.

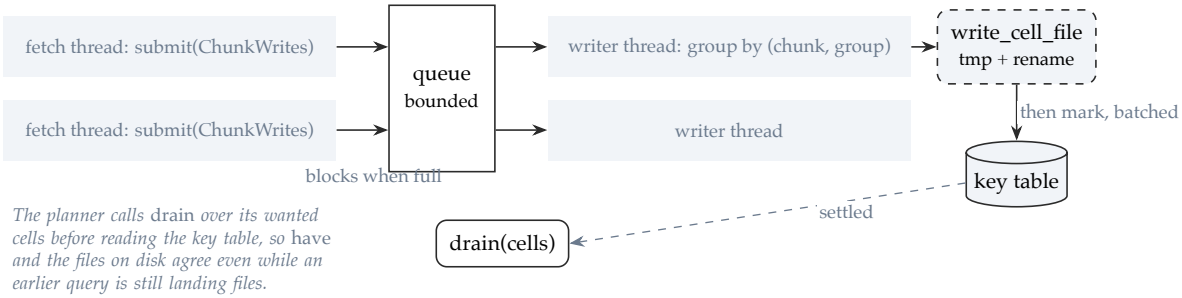


Figure 25: The writer. A bounded queue gives backpressure; marking after landing gives crash safety; the drain barrier gives read-after-write consistency to the next plan.

One primitive. Every lake file lands through one function: write to a temporary name in the same directory, then rename. The rename is atomic, so a concurrent reader never sees a partial file; it creates or replaces a directory entry, which is what the cell-statistics cache keys on; and it is the single place a test can block, slow or fail to exercise ordering.

The writer. Files are written by a small pool of threads fed by a bounded queue (Figure 25). The executor's threads block when the queue is full, so the fetch rate is throttled to the write rate and memory stays bounded; this is why the writer is not made "pure" by handing writes back to the caller as values, which would lose the backpressure. Two ordering rules are the contract: a key is marked present only *after* its file is on disk, so a crash loses cache and never truth; and a barrier exists that waits until every queued write over a set of cells is on disk and marked, which the planner calls before reading *have*, so a second query over the same area never reads a half-written lake. Marks are batched into one database transaction per flush because opening the key table costs 150 ms.

Failure unit. A group's chunk may touch several cells, and one cell's file may fail. The write returns, per cell, whether its file landed; the writer marks every beam of the group present in the cells that landed and in none of the cells that failed, so the chunk is re-fetched there rather than believed. Cells with no valid points are marked too, or an all-fill cell would be re-fetched forever.

Eviction. The lake has a byte budget per mission and globally. Eviction is by cell directory, least recently used, protecting the cells of the query in flight, and runs off the request thread. The cell statistics it needs are cached per directory and invalidated by the directory's modification time, which the rename updates.

what exists today. `lake.write_cell_file`, `write_chunk_group` (returns `{cell: landed}`), `ChunkWrite.group`, `_Writer` with `submit`/`drain`/`_flush`, `mark_ingested_many`, `ingested_chunk_cells`; `meta.duckdb` beside the index. ATL03 photons still write inline from their worker processes via `write_photons`.

19 What reads the lake

The lake is the system's contract with analysis (Figure 26). Every query of Chapter 4 beyond Q1 is arithmetic over lake files: the coverage survey (Q2) counts passes, days and years per cell from the lake's key table and the index manifest without touching the archive; co-registration (Q3) reads two sources' cells, converts each to a common frame and epoch with the conversions of Chapter 2, and pairs points within 35 m; the per-cell

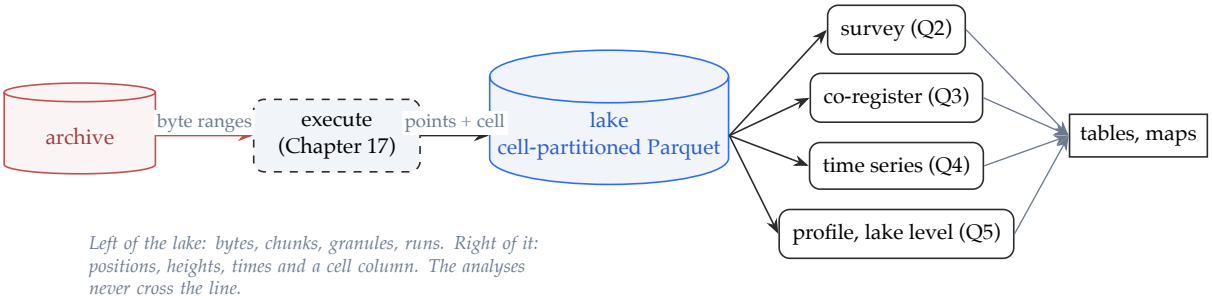


Figure 26: The lake between the retrieval engine and the analyses. Everything to its right is pure arithmetic over cell files; everything to its left is the machinery of Parts IV–VI.

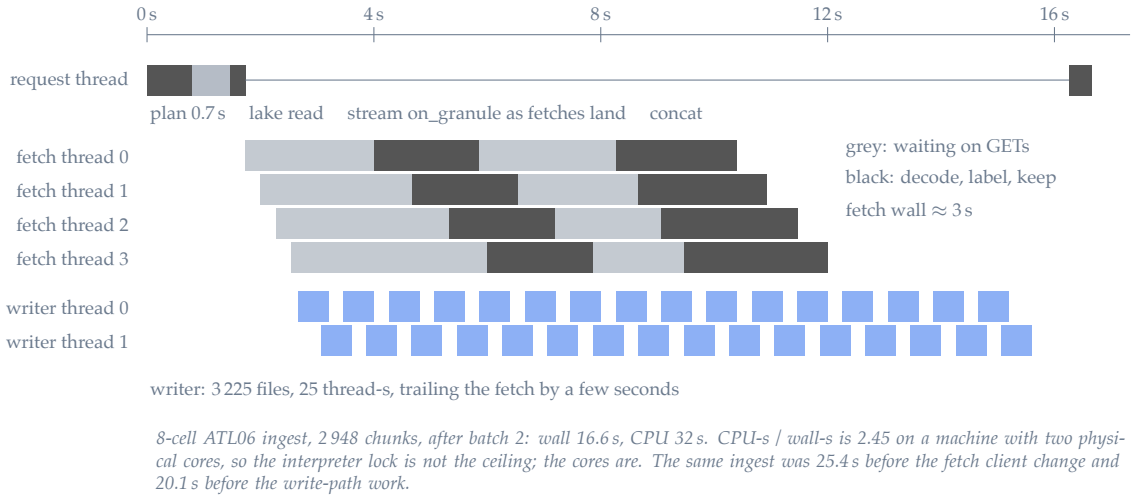


Figure 27: One query's threads over time. Fetch threads alternate between waiting and decoding; the writer trails; the request thread streams each granule's points as its fetch lands rather than after the slowest.

time series (Q4) reads every source's cells at resolution 8, fits its fixed-effects surface and reports window medians and a trend; the lake-level and profile tools (Q5) read a thin polygon's cells per pass. None of them knows a byte range, a chunk or a granule; they know a table of positions, heights, times and a cell column, per source, with the source's native height and the quality filter already applied. That is why the index and the lake share one resolution: the cell a row was addressed by is the cell its points are filed under, and an analysis that asks for a cell gets exactly the points the index promised.

20 The query at runtime

Four kinds of thread and one kind of process take part in a query (Figure 27). The request thread plans, reads the lake, and streams results to the caller as they arrive. A pool of fetch threads runs the executor of Chapter 17, one granule per task; it is sized for latency (up to 16) because its threads wait. The writer threads land files. For ATL03 only, decoding and the co-registration of 100,000 photons per chunk is CPU work heavy enough to be sent to spawned worker processes, one per (granule, beam) group, which fetch, decode, assign and write their own files and return the key set for the parent to mark.

Where the time goes, and what it rules out. Fetch wall is about 3 s for a scene: the archive client sustains 670 to 840 MB/s in-region and the request count is set by coalescing. The rest is CPU: decode, the remaining per-point masks, Arrow writes. The ratio of CPU-seconds to wall-seconds is 2.45 on two physical cores, which answers the question that always comes next. A process pool for decode and write would only help if the interpreter lock were serialising the threads, which would show as a ratio near 1.0 with wall well above fetch; at 2.45 the threads already overlap and the machine is simply out of cores. The ATL03 process pool is justified by its heavier per-chunk CPU, and it is spawned, not forked, because it runs inside a web server that holds a live archive client, an event loop and the writer's threads, none of which survive a fork intact.

What was measured not to help. Pre-caching the cells adjacent to the request (writing the whole decoded strip rather than the wanted cells) was tested twice. It made the first query 1.9 to 2.1 times slower and the neighbouring query slower too, because a chunk is re-fetched if *any* wanted cell is missing and the strip only brushes the neighbour. Raising the archive client's connection pool did nothing: the throughput cap is the machine's. Both results are recorded where the knobs live so the hypotheses are not re-tested by accident.

what exists today. `access.map_as_completed` (completion-order streaming), `FETCH_WORKER_CAP = 16`, `lake._Writer`, `planner._ensure` with a spawn `ProcessPoolExecutor`; the measurements in the `precache_adjacent` and `open_granule` docstrings and in `scripts/bench_ingest_phases.py`.

Part VIII

The framework

21 The collection descriptor

Parts V–VII described one build and one query, and this chapter discharges **R7**. Today there are six of each, and they differ in exactly the ways Table 3 listed: how a file's structure maps to element intervals, where positions come from, how a chunk is decoded into points, what quality means, which group a beam belongs to. Everything else (the driver, the pool, the resume set, the claim, the plan, the executor's loop, the keep predicate, the writer) is the same sequence written several times with those differences inlined. The framework is the step of making the differences a *value*, the `Collection` descriptor, and the sequence a single implementation that reads the value where it needs to (Figure 28).

```
@dataclass(frozen=True, slots=True)
class Dataset:
    key: str            # index-row and points column name
    path: str           # HDF5 path relative to the group, or a CSV field index
    role: str = ""      # "lat" | "lon" | "h" | "t" | "quality" | ""

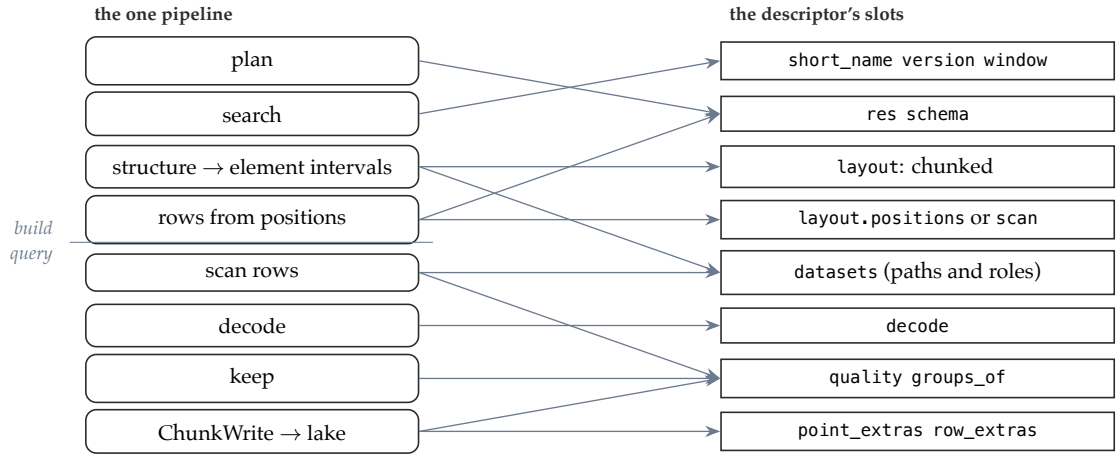
@dataclass(frozen=True, slots=True)
class HDF5Chunked:      # ATL03, ATL06, GLAS: one chunk map per group
    groups: tuple[str, ...]          # beams; ("",) for GLAS
    reference: str                   # the dataset whose chunking is the group's
    positions: Callable              # (group, datasets, fetch) -> Positions
    group_scalars: Callable | None   # (file) -> {"sc_orient", "sdp_epoch"}

@dataclass(frozen=True, slots=True)
class HDF5Spans:        # GEDI: rows are spans at the union of boundaries
    groups: tuple[str, ...]; positions: Callable; quality_field: Callable

@dataclass(frozen=True, slots=True)
class TextLines:        # ICESSN, GPS: rows are line byte spans per cell
    scan: Callable      # (bytes) -> LineScan(lat, lon, start, end)
    header_in_rows: bool

@dataclass(frozen=True, slots=True)
class Collection:
    key: str; mission: str; short_name: str; version: str
    res: int; schema: str
    window: tuple[str, str] | None
    layout: HDF5Chunked | HDF5Spans | TextLines
    datasets: tuple[Dataset, ...]
    parse_name: Callable[[str], dict]           # orbit numbers and dates from
     the name
    date_of: Callable[[str, dict], str]         # "YYYYMMDD" for window and
     manifest
    decode: Callable                            # (raws, row) -> Points, pre-
     mask
    quality: Callable | None                    # (Points) -> bool mask
    groups_of: Callable[[dict], Mapping[str, str]]   # beam -> "strong" | "weak"
      ...
    row_extras: tuple[str, ...] = ()            # extra index columns (GPS
     geometry)
    point_extras: tuple[str, ...] = ()          # extra lake columns ("quality",
      ...)
    job_params: Callable | None = None          # (record, io) -> (params, sha);
      GPS
```

Every function-valued field is a module-level function, so the descriptor



An edge means "this stage reads this slot". The stages on the left exist once. Adding a collection adds a column of values on the right and, at most, two small functions (`positions` or `scan`, and `decode`). No driver, no ingest loop, no resume set.

Figure 28: The plug board: which stage of the one pipeline consults which slot of a collection's descriptor. This is the structural meaning of "pluggable".

pickles across a spawn boundary as a name and a worker looks it up in a registry. Presentation (labels, colours, default selection) is not in the descriptor; it is an application table keyed by `Collection.key`, because the framework never draws anything.

Two descriptors, written out. ATL06, the reference HDF5 collection:

```
ATL06 = Collection(
    key="ATL06", mission="ATL06", short_name="ATL06", version="007", res=5,
     schema="atl06-3",
    window=("2018-10-01", "2027-01-01"),
    layout=HDF5Chunked(groups=("gt1l","gt1r","gt2l","gt2r","gt3l","gt3r"),
     reference="latitude",
        positions=slots.positions_identity,      # lat/lon per element
        group_scalars=slots.icesat2_scalars),    # sc_orient, sdp epoch
    datasets=(Dataset("latitude", "land_ice_segments/latitude", "lat"),
              Dataset("longitude", "land_ice_segments/longitude", "lon"),
              Dataset("h_li", "land_ice_segments/h_li", "h"),
              Dataset("delta_time", "land_ice_segments/delta_time", "t"),
              Dataset("atl06_quality_summary", "land_ice_segments/
     atl06_quality_summary",
                      "quality")),
    parse_name=slots.parse_icesat2_name, date_of=slots.date_from_name,
    decode=slots.decode_shared_chunk, quality=slots.quality_equals_zero,
    groups_of=slots.icesat2_strong_weak, point_extras=("quality",),
)
```

ICESSN, a text collection:

```
ICESSN = Collection(
    key="ICESSN", mission="ICESSN", short_name="ILATM2", version="2", res=5,
     schema="icessn-4",
    window=None, layout=TextLines(scan=icessn.scan_lines, header_in_rows=False),
    datasets=(), parse_name=icessn.parse_name, date_of=slots.date_from_name,
    row_extras=("itrf_year",), decode=icessn.parse_span_points, quality=None,
    groups_of=slots.no_groups,
    point_extras=("sn_slope", "we_slope", "itrf_year"),
)
```

GPS differs from ICESSN in four fields: `header_in_rows=True`, five row extras carrying the sled geometry, a `job_params` that reads the traverse-metadata granule once in the parent and hashes it, and its own `decode`.

The same pattern that stamps a parameter hash into a job key (Chapter 12) is what makes a metadata change invalidate the right files.

Where today's code goes. The per-collection functions that exist today become the slots; nothing is rewritten, only moved and given a signature (Table 11).

slot	existing code
`positions` (ATL03)	the segment-to-photon mapping in the ATL03 builder
`positions_identity`	ATL06 and GLAS position reads via `read_via_chunks`
`HDF5Spans` cutting	`index_gedi.chunk_spans`, `chunk_of`, `span_offset` (already pure)
`TextLines.scan`	`index_icessn._parse_fields` and its loop; the GPS equivalents
`decode`	`index_atl06._decode_chunk`, `index_glas._decode_chunk`, `index_gedi._decode_span`, `planner._decode_photons`, the two CSV parsers
`quality`	the `quality==0` masks, `index_gedi.quality_field`, GLAS's saturation cap
`groups_of`	`index.strong_beams` and GEDI's power flag
`job_params`	`build._gps_prepare`, `resolve_geometry`
row emission	the per-(chunk, cell) loops in every builder, written once

Table 11: Existing functions that become descriptor slots.

what exists today. `build.BuildCollection` is the build half of this descriptor (key, short name, version, resolution, index directory, resume set, timeout, workers, prepare); `coverage.collections()` holds the presentation half. The query half and the layout kinds are proposed.

22 Configuration as a value

The pipeline has about two dozen tunables: fetch mechanism, coalescing gap, thread and worker counts, lake writers, byte budgets, data directory, token location. Today they are read from the environment at about thirty sites, some at import time, which means a test, a bench and a worker process each discover the configuration by accident. The design makes configuration a frozen value built once at each *door* of the process and passed down (Figure 29).

```
@dataclass(frozen=True, slots=True)
class Config:
    data_dir: Path
    in_region: bool; s3_mechanism: str; coalesce_gap: int; crt_gbps: int
    fetch_workers: int; fetch_threads: int
    index_workers: int; job_timeout_s: int; start_method: str = "spawn"
    whole_object_max_bytes: int = 256 << 20
    lake_writers: int; lake_async: bool; lake_max_bytes: int
    precache_adjacent: bool
    token: str | None; edl_file: Path
    per_collection: Mapping[str, Overrides]   # e.g. GEDI job_timeout_s=600
    now: Callable[[], datetime] = utcnow                  # an injected clock

Config.from_sources(env, cli, file) -> Config   # cli > env > file > defaults
```

The environment variable names stay as they are, so deployment files are unchanged; what changes is that below the door nothing reads them. Customisation is by argument, not by subclass or monkeypatch: a different lake layout is a different `Layout` value passed to the lake; a different coalescing policy is a different function passed to the fetcher; a test's fake archive is a `fetcher` argument to the executor. The one transitional concession is a process-wide accessor returning the config set at the door, which old modules call until they are converted, and which an AST test forbids in the new packages.

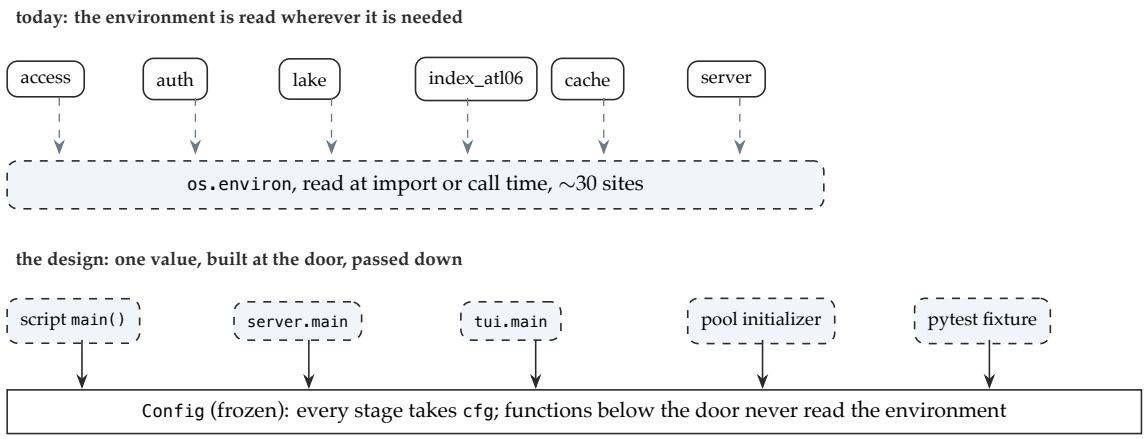


Figure 29: Configuration moves from scattered environment reads to one value built at each process door. A transitional process-wide accessor lets old modules read it until they take `cfg` explicitly.

23 The grid as a parameter

Part III compared four grids and chose H3 on three properties; this chapter says exactly what the design needs from a grid. The honest list is short, and the performance contract decides what may be abstracted at all.

```
@dataclass(frozen=True, slots=True)
class Grid:                          # one value per implementation
    name: str                        # "h3" | "s2" | "rhealpix"
    resolutions: range
    convex_cells: bool               # may the exact-run shortcut be trusted?
    cell_area_km2: Callable[[int], float]
    cells_vectorized: Callable[[Arr, Arr, int], Arr]       # lat, lon -> ids:
     THE hot kernel
    cells_for_area: Callable[[Ring, int], Sequence[int]]   # polyfill, overlap
     semantics
    parent: Callable[[int, int], int]
    children: Callable[[int, int], Sequence[int]]
    compact: Callable[[Sequence[int]], Sequence[int]]      # the claim's
     representation
    boundary: Callable[[int], Sequence[tuple[float, float]]]
    neighbors: Callable[[int, int], Sequence[int]]
    to_text: Callable[[int], str]; from_text: Callable[[str], int]
```

Three rules follow (Figure 30). The per-point kernel must be native; a grid without one is rejected, not tolerated. Per-cell operations run hundreds of times per query and may be plain Python. Storage ids are 64-bit integers whatever the grid's native form, because the index scan, the lake partitions and the manifest all rely on a sortable scalar; a grid whose ids are strings (rHEALPix's `"N2345"`) packs them into 64 bits in its module and exposes the text form for pages and the API.

The exact-run shortcut of Chapter 10 rests on one geometric fact, that a straight track meets a cell in one contiguous stretch, which holds for convex cells. H3 hexagons, S2 cells and rHEALPix cells are convex; a grid that is not sets `convex_cells` false, which disables the fast path and keeps the per-point fallback.

The grid is part of the store's identity: an index and a lake built under one grid are never read under another, so the grid's name is recorded



Figure 30: The three cost tiers of grid operations. Only the innermost is performance-critical, and it is the one operation a grid implementation must provide natively; a descriptor validates it against a time budget on 10^5 points at import.

beside them and carried in job keys and plans. Column and partition names drop the grid's name (`h3_cell` becomes `cell`), a one-time rename that rides with a schema bump. The addressing resolution becomes per grid in the descriptor, chosen by the same rule as today, a cell about the footprint of a chunk: H3 5, S2 9, rHEALPix 4 for ATL06.[4]

[4] Adding S2 is a module plus the C++ bindings' vectorised kernel. rHEALPix has no vectorised kernel in the ecosystem; it is the case that would justify a small Rust extension, which this structure makes a 200-line package rather than a change to the framework.

24 Replay

Three values make the system replayable, and each pays for itself in a different place (Figure 31).



`SearchRecord`: the job list recomputed without the catalogue. `QueryPlan`: an ingest re-executed without planning. Recorded bytes: decode and planner tests on real chunks without the archive.

Figure 31: Record once, replay without the effect.

- **Build.** A stored `SearchRecord` plus the current descriptor and the store's keys recomputes the job list exactly. A failed build is re-run from its record; a staleness policy on the record's timestamp decides when to search again. The build map that an explainer page reconstructs by hand becomes `render(record)`.
- **Query.** A `QueryPlan` serialises to a directory (the query as JSON, the rows as Arrow) and `execute(load(plan))` replays it. The ingest benchmark becomes "record once, execute N times with a different writer count", which is the honest form of the measurement Chapter 20 relied on.
- **Bytes.** A recording fetcher wraps any fetcher and writes each (url, offset, size) request's bytes to a fixture directory. One small granule per collection, recorded once in-region, lets decode and planner tests run on real bytes instead of synthetic blobs.

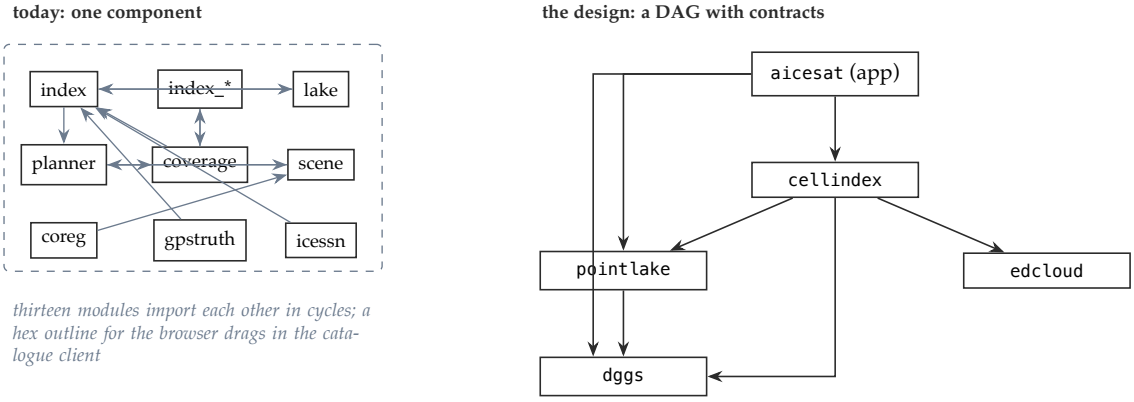


Figure 32: Before and after. The cycles of today's single component become a four-package DAG under the application, with layer contracts inside the framework package.

25 Packages and the structural view

The code today is one package of 13,000 lines whose index, lake, planner, coverage and collection modules form a single strongly connected import component of thirteen modules. The design splits it along the two seams the application crosses without the framework (grid geometry and the point cache) and keeps the rest as layers inside one framework package, separated by import contracts rather than by packaging (Figure 32, Table 12).

package	lines	responsibility	who else would use it
`dggs`	∼400	grid geometry over areas and cell sets; `Grid` values for H3 (and later S2)	anyone indexing by a global grid
`edcloud`	∼900	Earthdata mechanisms: login, catalogue search to `SearchRecord`, range and whole-object GETs, HDF5 chunk maps, decode	any project reading archive granules by byte range
`pointlake`	∼900	cell-partitioned Parquet memo, key table, writer, eviction	any cell-keyed point cache
`cellindex`	∼2 500	the framework: descriptor, values, build and query pipelines, store, claim, manifest	another altimetry or lidar project with its own descriptor
`aicesat`	the rest	scenes, analysis, API, server, TUI, browser, deploy	not reused

Table 12: The packages. One repository, one uv workspace; separate repositories only when an external consumer needs a different release cadence.

The mapping from today's modules is direct: `access` becomes `edcloud` (with the pure parts, coalescing and decode, separated from the mechanisms); `lake` becomes `pointlake`; `planner`'s geometry becomes `dggs`; `index`, `index_*`, `build`, `pool`, the rest of `planner` and the non-presentation half of `coverage` become `cellindex`; the analysis modules (co-registration, time series, lake levels, DEM and imagery readers) stay in the application, already pure at the function level, until a second consumer appears.

26 Purity in Python, and what not to do

Python enforces none of the above. The design is kept honest by tools that run in the one check target, since there is no continuous integration today:

1. **Import contracts** (import-linter): the layer order inside `cellindex` and the forbidden imports listed under Figure 32; the application may not import the build runner from request handlers.
2. **An AST purity test** over the pure module list, failing on environment reads, file opens, clocks, randomness, caches, threading and any module-level mutable assignment. The test that today forbids the query path from reaching the catalogue is this test's ancestor and is already parsed, not grepped.
3. **Types**: strict checking on the value modules and the grid package from day one, ratcheted elsewhere.
4. **Lint rules** that catch the usual leaks: naive clocks, prints in libraries, mutable defaults.
5. **Frozen, slotted values** with validation in `__post_init__` rather than defensive code downstream.
6. **Coverage floors** per package, and the golden suite (nine numbers against a cloned data directory) as the behavioural floor for the application at every migration step.

And the list of things the design deliberately does not do, each of which was considered:

- No effect system. Values are used in the four places replay or parallelism pays (`SearchRecord`, `Job`, `QueryPlan`, `ChunkWrite`); everywhere else a function that takes `cfg` and `io` arguments is the whole pattern.
- No class hierarchy and no plugin entry points for collections. A frozen dataclass with function-valued fields, one module per collection, pickles across spawn and is all that is needed.
- No storage interfaces over DuckDB, Parquet or the grid kernel. They *are* the performance; the pluggable points are the fetch mechanism, the layout value, the cell kernel and the coalescing policy, each a plain argument.
- No repository split, package index or independent versioning before a second consumer exists.
- No "pure" writer. The queue's backpressure is what bounds memory.
- No refactoring of the leaf kernels (decode, coalesce, span cutting, the parsers). They are the parts that are already right.
- No trust in purity by convention. Hence the list above.

Appendices

A Migration path

Each step lands on the main branch, passes the full suite and the golden suite, and deploys. Steps are ordered so that each removes a hazard on its own (Table 13).

step	what	retires	effort	status
A	`Config` at the doors; `from_sources`; `cfg` threaded through the fetcher, lake and planner; transitional accessor	environment reads below the doors	2 d	
B	`GranuleRef` and `Job`; one `run_build`; spawn pool with deadline and retry; resolution guard in every builder	five drivers, five fork pools, per-task pickling of the cell set	3 d	partly landed 2026-10-17; `SearchRecord` and the pool initializer remain
C	`SearchRecord` stored; the claim as a fold writing today's file shape	claim mutation, claim invalidation, the catalogue pickle	2 d	
D	content-addressed index store; manifest rows carry keys; glob fallbacks deleted; explicit conservative GC; one-shot rename on the box	six resume sets, footer provenance reads, the sidecars (the sha is in the key)	4 d	highest risk: 120k files renamed; rehearse on the clone
E	`Collection` descriptor and one `build_job` with three layouts; `Grid` enters here	six builder bodies, per-collection version constants	3 d	
F	`QueryPlan`, one `execute`, one `keep`; the lake takes a cell column	five fetch loops, five keep predicates, the ATL03 variant	5 d	seeds landed: one cell pass, group files, runs
G	extract packages; uv workspace; import contracts; delete the accessor	module singletons as cross-package state	3 d	
H	application split: scenes, analysis, admin; presentation registry; TUI over the public API	duplicated mission tables	4 d	

Table 13: About 26 days of focused work; D and F carry the least trustworthy estimates.

B Test strategy

Table 14 gives the shape of the tests per package and the coverage floor each must hold.

package	pure-core tests	interpreter tests	property tests	floor
`dggs`	literal geometry; claim compaction	none	compact and membership preserve every cell; polyfill covers centre-contained cells; the kernel agrees with the scalar API and meets the time budget	100 %
`edcloud`	coalesce, decode, span cutting on literals and recorded blobs	mechanisms behind a fake archive; catalogue with fixtures; token parsing	coalesce covers every input and respects the gap; decode∘encode identity with shuffle and gzip	95 / 80 %
`pointlake`	cell tables with a cell column; dtype preservation	writer ordering, eviction, stats, query scoping under a temporary config	write then query returns the same multiset	90 %
`cellindex`	jobs from literal records; rows from positions; plan on Arrow literals (todo ∩ have = ∅, every wanted cell is in lake or todo); keep; claim fold	`build_job` with a fake io (in-memory HDF5 plus recorded blobs); `run_build` with an inline pool; `execute` with a fake fetcher, byte-identical to the direct fetch for all six collections	job key stable under field order; plan idempotent	95 / 85 %
`aicesat`	science modules as they are	API, server, stream, TUI as they are; golden suite unchanged at every step		85 %

Table 14: Per-package test shape and coverage floors. Of today's ∼10 500 test lines, about 1 900 move with their modules, about 1 200 retire with what they tested, and the rest stay in the application.

C Measurements the design leans on

Table 15 collects every number the text relies on, with the instrument that produced it.

measurement	before	after	instrument
ATL06 build, 100 granules, 8 workers	44.7 s	21.3 s	`bench_build_io.py`
ATL06 build, workers 4 / 8 / 16 / 32 (before whole object)	81 / 42 / 26 / 26 s		`bench_build_io.py`
ATL06 per granule, serial reads through the block cache	66 reads, 2.0 s		profile
ATL06 full regional rebuild, 33 484 granules	3.9 h	62 min	staging rebuild
GLAS full rebuild, 4 799 granules	10 min	3.5 min	staging rebuild
ATL03 structure reads per granule at 1 / 4 / 16 MiB blocks	114 / 7 / 4		`bench_build_io.py`
16 spawned workers, peak memory		3.4 GB	`free -m`
8-cell ATL06 ingest, wall	25.4 s	16.6 s	`bench_ingest_fresh.py`
same, CPU-seconds	40.7	32.4	idem
same, CPU-s / wall-s (2 physical cores)		2.45	idem
same, lake files written	6 897	3 225	idem
same, cell assignment thread-seconds	24–31	0	idem; 2 948 of 2 948 chunks from runs
exact runs on real ATL06 rows (12 granules)		99.3 %	`tests/test_index_cell_runs.py`
ATL03 plan for a 0.3° box	36 s	0.66 s	query trace
index store with sidecar provenance	7.5 GB	4.7 GB	`migrate_index_footers.py`
in-region fetch throughput, CRT client	300–400	670–840 MB/s	`bench_coalesce.py`
precache-adjacent, first box / neighbour (on vs off)	16.8 / 13.9 s	31.9 / 21.7 s	`bench_ingest_fresh.py`
catalogue hits for a 2° box, ATL06 / ATL03 / GLAS	549–571 / 560–602 / 93–139		`measure_search_modes.py`
bytes moved by whole-granule retrieval vs needed, ATL06 / ATL03	63–80× / 57–94×		idem
same, the 15 km transect box, ATL06 / ATL03	86× / 475×		idem
catalogue hits with no point in the box's cells, ATL06 / GLAS	9.6–31.8 % / 8.6–32.3 %		idem
granule footprint area over box area, ICESat-2, 2° box / transect	7.2–9.8× / 852–1 009×		idem
ATL06 granule size over Greenland, quartiles	72 / 88 / 96 MB		idem
ATL03 granule size over Summit, quartiles	0.95 / 1.4 / 2.7 GB		idem
ATL03 staging rebuild, 48 838 granules, 16 workers, 4 MiB blocks	12 h	7.4 h	staging rebuild, 2026-10-18

Table 15: Measurements on the deployed instance (m6i.xlarge, 4 vCPU, 15 GB, us-west-2), 2026-10-16 to 19. The search-mode rows are the full table of `docs/notes/search-modes-2026-10.md`.

D Glossary

along-track data ordered by time along a ground track, with position stored per measurement.

footprint (of a measurement) the ground patch one laser pulse illuminates; (of a granule) the catalogued extent.

granule one archive file: an orbit region, an orbit arc, minutes of a flight, a survey.

RGT, cycle, region ICESat-2's reference ground track (1 of 1 387), its 91-day repeat count, and one of the 14 latitude cuts of an orbit.

campaign a GLAS laser operating period (19, L1A–L2F) or an IceBridge season.

platelet one ILATM2 record: a plane fitted to a block of ATM points, 80 m across at nadir.

DGGS discrete global grid system: nested partitions of the Earth with an identifier scheme.

aperture the number of children a grid cell has at the next resolution (H3: 7).

polyfill the set of cells an area overlaps.

compaction replacing every complete set of children in a cell set by their parent, recursively.

over-claim a catalogued footprint larger than the ground the granule measured; a false positive is a search hit with no point in the area.

chunk a fixed-size, separately compressed piece of an HDF5 dataset; the unit of a byte range.

chunk map (offset, size, filters) for every chunk of a dataset, read from the file's B-tree once at build.

span GEDI's row unit: an element interval inside one chunk of every dataset.

cell a grid cell at some resolution; addressing cells key rows and lake files, claim cells record coverage.

run the element interval of one cell inside one chunk, and whether it is pure.

index one Parquet file per granule of (chunk, cell) rows carrying byte ranges and runs.

claim the compacted cell set a collection's index is known to cover; consulted before every query.

manifest the fold of an index into distinct (cell, granule, month) rows; names the files a query opens.

provenance the filter set a granule's file was built against, recorded in its footer by hash.

lake the cell-partitioned Parquet memo of decoded points, read before and written after a fetch.

beam group the set of beams the planner requests together and the lake files together.

have, todo the chunks a plan serves from the lake, and those it fetches.

door a process entry point where configuration is built once: a script's main, the server, the TUI, a pool initializer, a test fixture.

slot a function-valued field of a collection descriptor that a pipeline stage calls.

value immutable data; a frozen dataclass here.

effect a function that touches the network, the disk, a clock or a pool.

References

[1] B. E. Schutz, H. J. Zwally, C. A. Shuman, D. Hancock, and J. P. DiMarzio. Overview of the ICESat Mission. *Geophysical Research Letters*, 32:L21S01, 2005.

[2] H. Jay Zwally, Robert Schutz, John DiMarzio, and David Hancock. GLAS/ICESat L1B global Elevation Data (HDF5), Version 34 (GLAH06), 2014.

[3] W. B. Krabill, W. Abdalati, E. B. Frederick, S. S. Manizade, C. F. Martin, J. G. Sonntag, R. N. Swift, R. H. Thomas, and J. G. Yungel. Aircraft laser altimetry measurement of elevation changes of the Greenland ice sheet: Technique and accuracy assessment. *Journal of Geodynamics*, 34(3–4):357–376, 2002.

[4] National Snow and Ice Data Center. Pre-IceBridge ATM L2 Icessn Elevation, Slope, and Roughness, Version 1: User Guide, 2019. Describes the icessn resampling: plane fits to blocks of ATM points, an 80 m across-track nadir platelet plus outer platelets, about 0.5 s of data per record.

[5] Michael Studinger. IceBridge ATM L2 Icessn Elevation, Slope, and Roughness, Version 2 (ILATM2), 2014.

[6] Thorsten Markus, Tom Neumann, Anthony Martino, Waleed Abdalati, Kelly Brunt, Beata Csatho, Sinead Farrell, Helen Fricker, Alex Gardner, David Harding, Michael Jasinski, Ron Kwok, Lori Magruder, Dan Lubin, Scott Luthcke, James Morison, Ross Nelson, Amy Neuenschwander, Stephen Palm, Sorin Popescu, C.K. Shum, Bob E. Schutz, Benjamin Smith, Yuekui Yang, and Jay Zwally. The Ice, Cloud, and land Elevation Satellite-2 (ICESat-2): Science requirements, concept, and implementation. *Remote Sensing of Environment*, 190:260–273, 2017.

[7] Thomas A. Neumann, Anthony J. Martino, Thorsten Markus, Sungkoo Bae, Megan R. Bock, Anita C. Brenner, Kelly M. Brunt, John Cavanaugh, Stanley T. Fernandes, David W. Hancock, Kaitlin Harbeck, Jeffrey Lee, Nathan T. Kurtz, Philip J. Luers, Scott B. Luthcke, Lori Magruder, Thomas A. Pennington, Luis Ramos-Izquierdo, Timothy Rebold, Jonah Skoog, and Thomas C. Thomas. The Ice, Cloud, and Land Elevation Satellite–2 mission: A global geolocated photon product derived from the Advanced Topographic Laser Altimeter System. *Remote Sensing of Environment*, 233:111325, 2019.

[8] Thomas A. Neumann, Anita Brenner, David Hancock, John Robbins, Jack Saba, Kaitlin Harbeck, Ann Gibbons, Jeffrey Lee, Scott B. Luthcke, and Tim Rebold. ATLAS/ICESat-2 L2A Global Geolocated Photon Data, Version 7 (ATL03), 2025.

[9] Benjamin Smith, Helen A. Fricker, Nicholas Holschuh, Alex S. Gardner, Susheel Adusumilli, Kelly M. Brunt, Beata Csatho, Kaitlin Harbeck, Alex Huth, Thomas Neumann, Johan Nilsson, and Matthew R. Siegfried. Land ice height-retrieval algorithm for NASA's ICESat-2 photon-counting laser altimeter. *Remote Sensing of Environment*, 233:111352, 2019.

[10] Benjamin Smith, Susheel Adusumilli, Beáta M. Csathó, Denis Felikson, Helen A. Fricker, Alex Gardner, Nicholas Holschuh,

Jeffrey Lee, Johan Nilsson, Fernando S. Paolo, Matthew R. Siegfried, Tyler Sutterley, and the ICESat-2 Science Team. ATLAS/ICESat-2 L3A Land Ice Height, Version 7 (ATL06), 2025. User guide: `https://nsidc.org/sites/default/files/documents/user-guide/atl06-v007-userguide.pdf`.

[11] Ralph Dubayah, James Bryan Blair, Scott Goetz, Lola Fatoyinbo, Matthew Hansen, Sean Healey, Michelle Hofton, George Hurtt, James Kellner, Scott Luthcke, John Armston, Hao Tang, Laura Duncanson, Steven Hancock, Patrick Jantz, Suzanne Marselis, Paul L. Patterson, Wenlu Qi, and Carlos Silva. The Global Ecosystem Dynamics Investigation: High-resolution laser ranging of the Earth's forests and topography. *Science of Remote Sensing*, 1:100002, 2020.

[12] University of Maryland GEDI Science Team. GEDI mission technology: Footprint and sampling geometry, 2024. 25 m footprints every 60 m along track on eight ground tracks 600 m apart. Accessed 2026-10-18.

[13] Ralph Dubayah, Michelle Hofton, James Bryan Blair, John Armston, Hao Tang, and Scott Luthcke. GEDI L2A Elevation and Height Metrics Data Global Footprint Level, Version 3 (GEDI02_A), 2024.

[14] Robert L. Hawley, D. J. Pickell, J. R. McConnell, T. A. Neumann, D. Felikson, and S. W. Dorsi. ICESat/ICESat-2 Traverse: Monthly GPS Surface Elevation Data at Summit Station, Greenland, Version 1 (IS2TGPSSS), 2026.

[15] The HDF Group. HDF5 File Format Specification, Version 3.0, 2025. Accessed 2026-10-18.

[16] NASA Earthdata. CMR Search — API Documentation, 2026. Accessed 2026-10-18.

[17] Luis López, Matt Fisher, Amy Steiker, Aaron Friesz, Qiusheng Wu, and the earthaccess contributors. earthaccess: a Python library to search for and download or stream NASA Earth science data, 2026. Accessed 2026-10-18.

[18] Kevin Sahr, Denis White, and A. Jon Kimerling. Geodesic Discrete Global Grid Systems. *Cartography and Geographic Information Science*, 30(2):121–134, 2003.

[19] Open Geospatial Consortium. Topic 21: Discrete Global Grid Systems — Part 1 Core Reference System and Operations, and Equal Area Earth Reference System. Technical Report OGC 20-040r3, version 2.0.0, Open Geospatial Consortium, 2021.

[20] Uber Technologies. H3: Hexagonal hierarchical geospatial indexing system — core library overview, 2025. Accessed 2026-10-18.

[21] Google. S2 Geometry Library: S2Cell hierarchy, 2025. Accessed 2026-10-18.

[22] K. M. Górski, E. Hivon, A. J. Banday, B. D. Wandelt, F. K. Hansen, M. Reinecke, and M. Bartelmann. HEALPix: A framework for high-resolution discretization and fast analysis of data distributed on the sphere. *The Astrophysical Journal*, 622(2):759–771, 2005.

[23] R. G. Gibb. The rHEALPix discrete global grid system. *IOP Conference Series: Earth and Environmental Science*, 34:012012, 2016.

[24] Apache Software Foundation. Apache Parquet: File Format, 2025. Accessed 2026-10-18.